



tapps-brain

Persistent Memory for AI Coding Agents

A Deep Architecture & Operations Reference

Call trees by function. Tool catalog and skill surface.
Agent-to-agent flows. System prompts and hooks. Decay mathematics.
Knowledge graph. 2026 literature anchor.

Edition 2.1 (verified) · Issued 2026-05-20 · v3 series · PostgreSQL-only plane (ADR-007)

Prepared by NLT Labs · tapps-brain engineering

Contents

Contents	2
1. Executive Summary	5
2. Why Persistent Memory Matters	6
3. The Three Layers of an Agent	7
3.1 Instruction layer — what the agent is told to do	7
3.2 Capability layer — what the agent can do	7
3.3 Memory layer — what the agent remembers	7
4. System Architecture Overview	9
5. C4 — System Context	10
6. C4 — Container View	12
7. C4 — MCP Server Components	14
8. C4 — Services Layer	16
9. Hexagonal — Ports and Adapters	18
10. Domain Data Model	19
11. Error Taxonomy	21
12. The MCP Tool Catalog	22
12.1 brain_* — agent facade	22
12.2 memory_* — full CRUD and search	24
12.3 kg_* — Knowledge Graph	24
12.4 hive_* — cross-agent sharing	26
12.5 feedback_* and diagnostics — the quality loop	26
12.6 agents_* — the agent registry	26
12.7 maintenance_* — operator only	26
12.8 memory:// resources and workflow prompts	26
13. The Skills Layer	27
13.1 Core memory skills	27
13.2 Linear and release skills (one-of-many integrations)	28
13.3 Specialist sub-agents — Ralph loop and TAPPS review	28
14. Hooks and System Prompts	29

14.1 The SessionStart recall primer	29
14.2 Other hooks in the standard configuration	29
14.3 Why not put 'always call brain_recall' in CLAUDE.md?	29
15. Profiles — Tier and Decay Shapes	31
15.1 Decay curves — exponential vs power-law	31
15.2 Profile comparison — three contrasting profiles	32
15.3 Importance-tag multipliers	32
15.4 Promotion and demotion	32
16. Call Tree — Recall	33
16.1 Function-level call tree	33
16.2 Worked example	34
17. Call Tree — Save / Remember	35
17.1 Function-level call tree	35
17.2 Phase-by-phase latency budget	36
17.3 Worked example	36
18. Agent-to-Agent Flows	37
18.1 Scope semantics	37
18.2 Hive change feed (LISTEN/NOTIFY)	37
18.3 Agent registry	37
18.4 Fleet topology — what an operator sees	38
18.5 Federation — cross-project sharing	38
19. Sequence Diagrams — Full Reference	39
20. Mathematical Foundations	42
20.1 Okapi BM25	42
20.2 Reciprocal Rank Fusion	42
20.3 Composite scoring	42
20.4 Exponential and power-law decay	43
20.5 FSRS-style stability and difficulty	43
20.6 Bayesian-style confidence update	43
20.7 Bloom filter false-positive bound	44
20.8 Jaccard / TF-IDF for deterministic consolidation	44
20.9 EWMA anomaly detector	44
20.10 pgvector HNSW geometry	44
21. The Knowledge Graph	45

21.1 Three table families	45
21.2 Evidence-required edges (ADR-012)	45
21.3 The five KG tools — when to reach for each	46
22. Data Plane and Security	47
22.1 Tables and indexes	47
22.2 STRIDE threat model	47
23. What Works in 2026 — Literature Anchor	49
23.1 Textbook foundations (still canon in 2026)	49
23.2 Recent techniques the literature has converged on	49
23.3 What the 2025–2026 community has decided not to do	50
23.4 Reference list	51
24. End-to-End Worked Example	52
25. Closing — Business Value Restated	54
A. Appendix — Module Dependency Overview	55
B. Appendix — Source Verification Log	57

1. Executive Summary

What tapps-brain is, what it eliminates, and how to read this document.

tapps-brain is a persistent, deterministic memory system for AI coding agents. It preserves the architectural decisions, code conventions, debug findings, and workflow patterns that an agent (and the team it serves) discovers over a sequence of sessions, and surfaces them — ranked, decayed, and de-duplicated — as cheap retrieval context at the start of every subsequent task.

The system is implemented as a single PostgreSQL-backed service. It exposes the agent-facing surface as **65 Model Context Protocol tools** across eight families (*brain_**, *memory_**, *kg_**, *hive_**, *feedback_**, *agents_**, *maintenance_**, plus *memory://* resources and workflow prompts), and provides a REST surface with ADR-enforced parity for clients that cannot speak MCP. Every operation in the recall hot path is deterministic: BM25 lexical scoring, pgvector HNSW dense retrieval, Reciprocal Rank Fusion, lazy exponential or power-law decay, deterministic Jaccard/TF-IDF consolidation. No LLM call participates in the request path.

How to read this document

Chapters 2–4 are the business case and high-level architecture (skim). Chapters 5–11 are the diagram-driven architectural reference (cite). Chapters 12–14 are the deep operational surface — the 65 MCP tools, the .claude skills layer, the SessionStart hook (read if you are wiring tapps-brain into a real agent fleet). Chapters 15–18 are the function-level call trees and agent-to-agent flows with concrete examples (read if you are integrating, extending, or debugging). Chapters 19–22 are the mathematical foundations, the data plane, security, and observability. Chapter 23 is the 2026 literature anchor — read this last when you need to explain the design choices to a skeptical audience.

Headline characteristics

Persistence	PostgreSQL 17 + pgvector HNSW + tsvector (ADR-007 — no SQLite)
Surfaces	MCP Streamable HTTP on /mcp, REST on /v1/*, Operator MCP on :8090
Tools	65 agent-facing tools + 8 memory:// resources + 3 workflow prompts
Profiles	6 built-in (repo-brain, customer-support, home-automation, personal-assistant, project-management, research-knowledge)
Skills + sub-agents	20 .claude skills + 11 specialist sub-agents (7 ralph-* + 4 tapps-*)
Tiers (repo-brain)	architectural 180 d / pattern 60 d / procedural 30 d / context 14 d half-life
Decay model	Power-law $\beta \approx 3.29$ by default in repo-brain (FSRS-anchored); exponential available per layer
Scale target	200+ concurrent agents per cluster, row-scoped (project_id, agent_id) isolation, MVCC concurrency, FORCE RLS
LLM cost in hot path	Zero (offline judges are opt-in for evaluation only)

2. Why Persistent Memory Matters

The business case — what changes when agents stop forgetting.

The unit economics of an AI engineering organisation are dominated by repeat work. A coding agent without persistent memory enters every session in the same naive state: the codebase looks foreign, the team's conventions are invisible, and the failure modes that were debugged yesterday are unknown today. The team compensates by re-supplying context — either by hand (engineer time) or by ballooning the prompt with retrieved files (model tokens). Both costs are paid *per session*, and both scale linearly with the number of agents.

Persistent memory shifts the cost curve. Once a decision is captured with its rationale, the next session retrieves it in a few hundred tokens instead of re-deriving it from a much larger working set. Once a debug session reveals a non-obvious invariant, every subsequent agent that touches the same module sees it. Once an architectural pattern is named, the team — human and machine — stops re-inventing local variants.

Outcomes

- **Lower per-task model cost.** A focused 1 500-token recall replaces a multi-thousand-token context dump on every turn. Token spend scales with task novelty, not with codebase size.
- **Higher first-pass correctness.** Agents arrive pre-loaded with the constraints, previous fixes, and architectural rules that govern the task. Fewer wrong turns, fewer review cycles, fewer regressions.
- **Onboarding curve approaches zero.** A new agent (or a new human teammate consuming the same memory through MCP) inherits the project's accumulated knowledge from minute one.
- **Institutional knowledge survives team turnover.** Rationale captured by one agent is durable, ranked, and reachable by any later agent — including the next model generation.
- **Audit-grade traceability.** Every memory write lands in an append-only audit log with an integrity hash, the responsible agent identity, and a timestamp. Compliance and incident reviews finally have a real artifact.
- **Vendor neutrality preserved.** Memory lives in your PostgreSQL, ranked by deterministic mathematics. Switching model providers does not invalidate the corpus.

3. The Three Layers of an Agent

What tells the agent what to do, what the agent can do, and what the agent remembers.

tapps-brain sits inside a three-layer agent stack. Conflating these layers is the most common source of integration confusion; separating them is the most important conceptual move in this document.

3.1 Instruction layer — what the agent is told to do

The instruction layer is everything the agent reads on every turn. It is static or session-static text injected at session start by the host harness. It tells the agent who it is, what its tools are, and how to behave. In a tapps-brain fleet this layer is composed of **three artefacts**:

CLAUDE.md AGENTS.md	/	Per-project instructions — coding conventions, build commands, the project's architectural narrative. Lives in the repo. Loaded by Claude Code on every session start.
SOUL.md		Per-container <i>workspace personality</i> — the agent's personality, tone, fixed operating rules. Lives at <code>/app/SOUL.md</code> in the project container. Distinct from project instructions: SOUL.md is who the agent <i>is</i> ; CLAUDE.md is what the project <i>requires</i> .
USER.md		Per-container <i>user preferences</i> — the operator's preferences for how to be communicated with. Also lives in <code>/app/</code> . Stable across sessions within one container.

These three files are deliberately **not** in tapps-brain. They are per-instance state; the brain is the durable cross-session store of **discovered knowledge**, not of personality.

3.2 Capability layer — what the agent can do

The capability layer is the agent's tool surface. In a tapps-brain-enabled fleet this is composed of:

- **The 65 MCP tools and 8 resources** served by the standard *tapps-brain-mcp* server (chapter 12).
- **13 operator tools** served by the separate *tapps-brain-operator-mcp* server on `:8090`, gated by a different auth token. The standard server *never* exposes these.
- **20 .claude skills** the harness loads (chapter 13). Skills are *tool compositions* — small specialised playbooks that route the agent to the right tool combination for a recurring task (e.g. *tapps-finish-task* runs `validate` → `checklist` → optional memory save in a fixed order).
- **11 specialist sub-agents** — 7 *ralph*-* for the autonomous-development loop (driver, architect, coordinator, explorer, tester, background-tester, reviewer) and 4 *tapps*-* for quality / review (researcher, review-fixer, reviewer, validator).

3.3 Memory layer — what the agent remembers

The memory layer is tapps-brain itself: the durable, ranked, decayed, agent-shared corpus of *discovered knowledge*. It is the only one of the three layers that persists across sessions, evolves with feedback, and is shared across agents and projects (via Hive and Federation).

The contract

Instructions (CLAUDE.md, SOUL.md, USER.md) tell the agent *how to act*. The tool catalog (MCP + skills) tells the agent *what it can call*. The memory (tapps-brain) tells the agent *what has been learned*. None of these three replaces the others. Most integration bugs reduce to putting content in the wrong layer — for example, saving project conventions as memories instead of writing them into CLAUDE.md, or expecting CLAUDE.md to evolve with feedback.

4. System Architecture Overview

Five subsystems, one shared persistence plane.

tapps-brain composes five subsystems behind a thin transport layer: an agent facade, a memory store, a retrieval pipeline, an optional cross-agent Hive, and a quality loop. All durable state lives in PostgreSQL. Agents never address Postgres directly; they call the agent facade or the MCP tool surface.

Agent facade	Five-method surface — remember, recall, forget, learn-from-success, learn-from-failure. Agents reason at this level only.
Memory store	Coordinates safety, conflict detection, write policy, persistence, propagation, consolidation, decay, and audit. Synchronous core; optional async wrapper.
Retrieval pipeline	Lexical (BM25 over tsvector candidates) and dense (pgvector HNSW) channels merged with Reciprocal Rank Fusion. Optional FlashRank cross-encoder rerank.
Hive (optional)	Cross-agent memory sharing inside a project. Declarative group membership, expert auto-publishing, LISTEN/NOTIFY change feed.
Quality loop	Deterministic feedback signals, EWMA-based anomaly detection, circuit breaker on retrieval quality, Bayesian confidence updates.

5. C4 — System Context

Who talks to tapps-brain, and across which boundaries.

Coding agents (Claude Code, Cursor, AgentForge, Ralph loop) speak MCP and REST. A single human operator role uses the CLI for maintenance. The brain itself is one container that owns a Postgres cluster, optionally emits OpenTelemetry traces, and optionally calls out to Context7 for library documentation lookups during validation paths.

tapps-brain — System Context

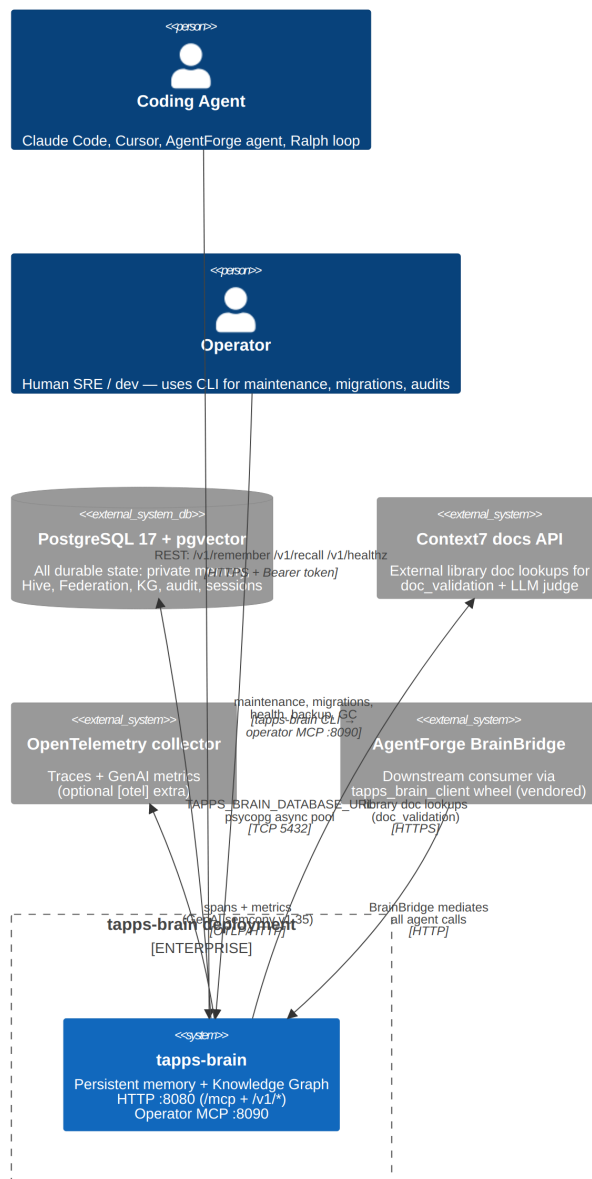


Figure 5.1 — C4 system context. Dashed boundaries are external; the brain runs as a single container with one Postgres dependency.

6. C4 — Container View

How a request lands inside the container and reaches Postgres.

Inside the container, a FastAPI ASGI adapter authenticates the caller, applies the profile gate, and dispatches either to the REST router or to the FastMCP streamable endpoint. Both transports terminate in the same set of pure service functions in `services/*.py`, which in turn talk to the memory store, the retriever, and the experience-event recorder. The experience-event recorder is the only writer that holds a multi-table transaction open — every other write goes through the memory store's write-through cache.



7. C4 — MCP Server Components

The agent-facing tool layer is a thin wrapper over the services package.

Tools are organised by capability family — brain, memory, knowledge graph, hive, feedback, maintenance, resources, agent registry. Every tool body delegates to a function inside *services/*; the MCP layer adds nothing but envelope deserialisation, profile gating, and JSON serialisation. This separation is what makes the REST and MCP surfaces enforceably equivalent.

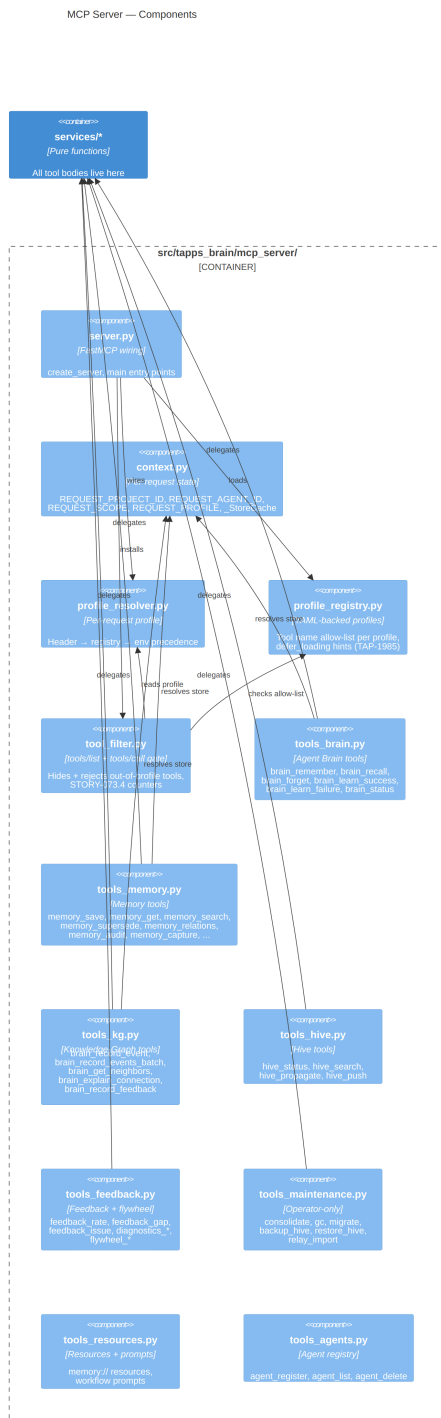


Figure 7.1 — MCP server components. The `tool_filter` enforces the per-profile allow-list at both `tools/list` and `tools/call`.

8. C4 — Services Layer

Pure functions, transport-agnostic, end-to-end testable.

The services package is the architectural seam of the system. Every business operation has exactly one implementation here; transports (REST, MCP, CLI, library) are thin parameter adapters. Services depend on protocols, never on transports.

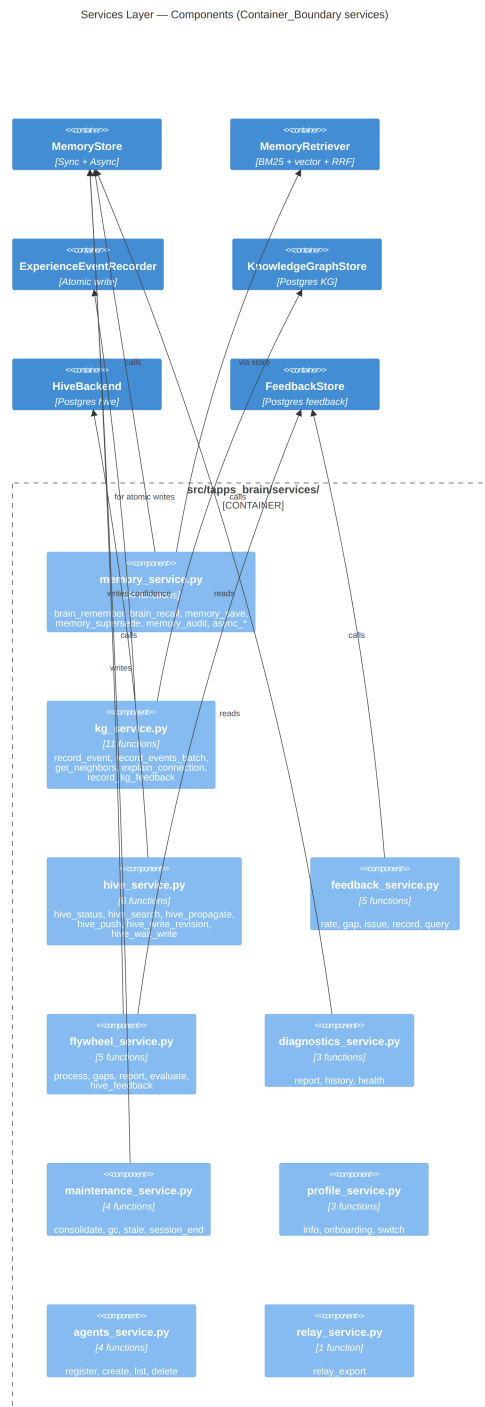


Figure 8.1 — Services layer composition. *memory_service.py* dominates the surface area; *experience.ExperienceEventRecorder* owns the atomic Knowledge Graph write path.

9. Hexagonal — Ports and Adapters

The architectural invariant the codebase enforces.

The core domain depends only on protocol definitions; every concrete adapter (PostgreSQL backends, FlashRank reranker, SentenceTransformer embedding provider, Context7 lookup engine, LLM judges) is wired in by factories that inspect configuration. This is what allows the codebase to remain testable without a database, swap embedding providers without touching retrieval logic, and add a new transport without rewriting the core.

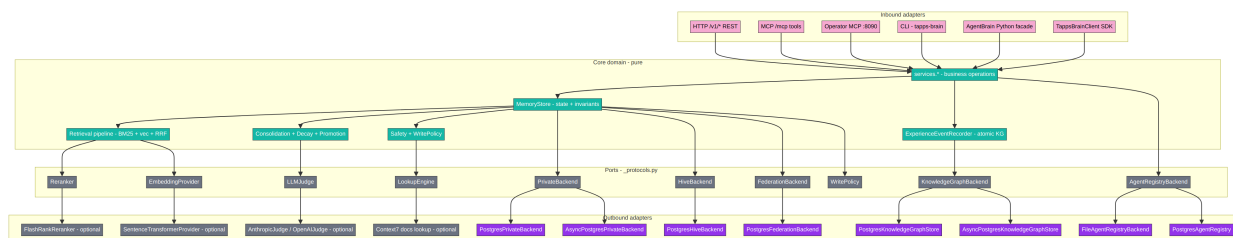


Figure 9.1 — Ports & Adapters. The core never imports from `tapps_brain.http.*` or any `postgres_*` module directly; it depends on the protocols defined in `_protocols.py`.

10. Domain Data Model

The MemoryEntry is the spine — every persisted row maps to one.

The Pydantic *MemoryEntry* carries every field an agent or operator ever needs to reason about a memory: the content, its tier, its confidence, its source attribution, its tags, its lifecycle status, its temporal validity window, its FSRS-style stability and difficulty, its integrity hash, and its agent scope. The Knowledge Graph views project the entity / edge / evidence tables into a recall-friendly shape — the same recall response can carry ranked memories, the entities mentioned, the edges that connect them, and the supporting evidence quotes, all in one document.

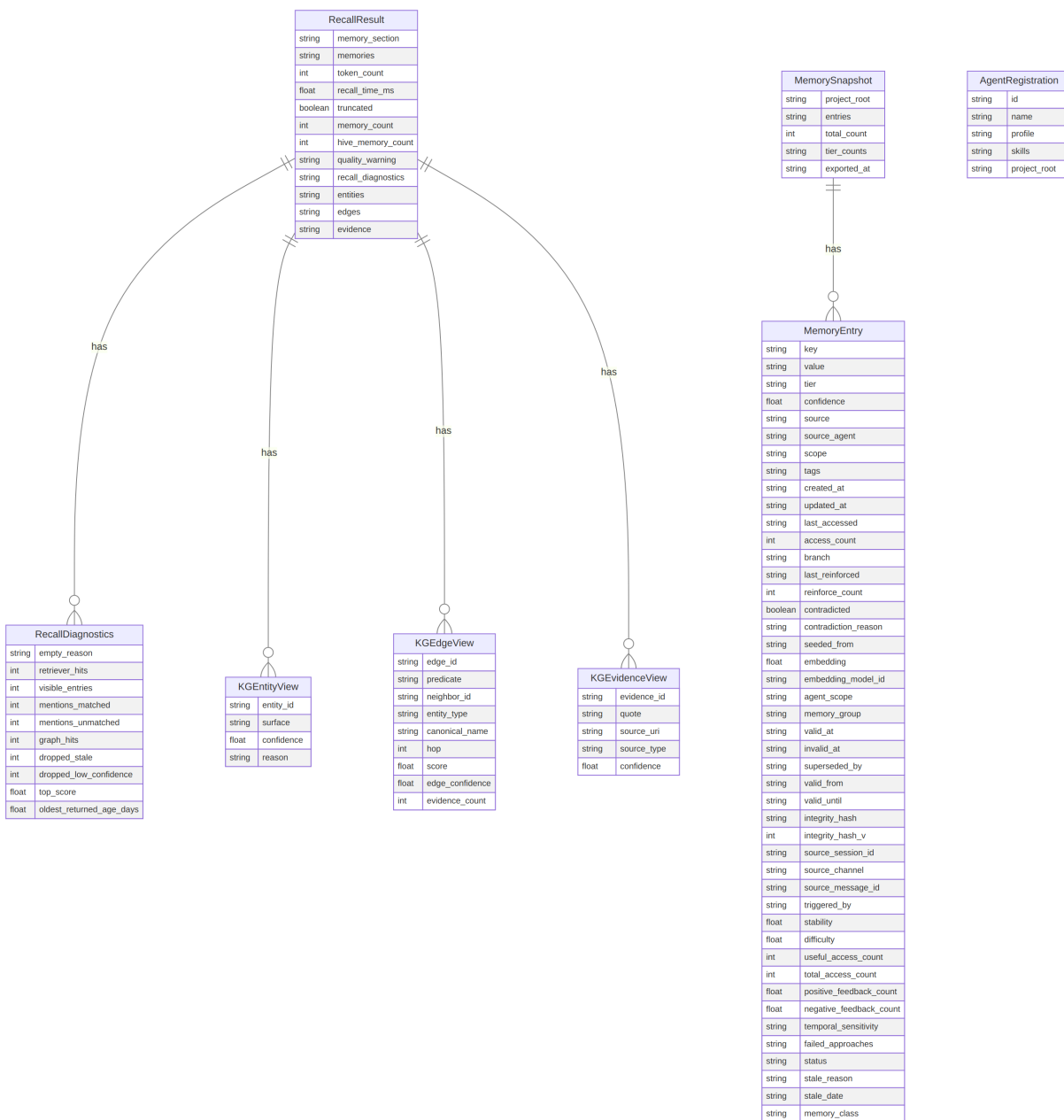


Figure 10.1 — Pydantic entity-relationship overview. *RecallResult* composes *MemoryEntry*, the KG views, and diagnostics into the single document returned to agents.

11. Error Taxonomy

Stable wire codes layered on a semantic SDK hierarchy.

Errors are first-class. The wire taxonomy maps each fault to a stable code, an HTTP status, a JSON-RPC code, and a retry policy. Clients catch by intent — auth errors stop and re-token, transient errors are retried by the SDK and escalated otherwise, transport errors degrade gracefully.

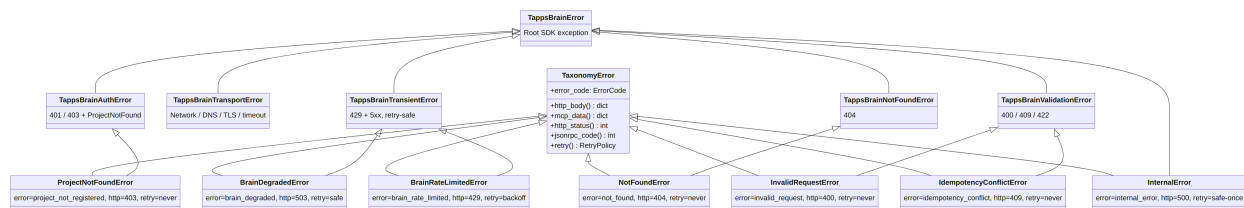


Figure 11.1 — Wire taxonomy (top half) layered over the semantic SDK hierarchy (bottom half). Catch by intent, not by string match.

12. The MCP Tool Catalog

65 tools, 8 resources, 3 prompts — what every family does and when to reach for it.

An agent's effective capability surface is whatever `tools/list` returns for its profile. tapps-brain ships **65 unique tools** across eight families (per the canonical manifest `docs/generated/mcp-tools-manifest.json` — the raw `@mcp.tool()` decorator count is higher because a handful of tools are cross-registered in both an agent-facing and an operator-facing file), **8 read-only resources** behind the `memory://` URI scheme, and **3 workflow prompts**. Tools are gated by an MCP profile (`agent_brain`, `coder`, `reviewer`, `full`, `operator`); each profile declares an *eager* subset returned by `tools/list` on session start and a *deferred* subset reachable only via Tool Search BETA.

MCP profile tool-visibility matrix (eager = returned by `tools/list` immediately; deferred = reachable via Tool Search BETA; — = hidden)

	agent_brain	coder	reviewer	full	operator
brain_* (facade)	eager	eager	—	eager	eager
memory_* (CRUD + search)	—	deferred	deferred	deferred	deferred
hive_* (cross-agent share)	—	deferred	—	deferred	deferred
kg_* (Knowledge Graph)	deferred	eager	—	deferred	deferred
feedback_* (signals + diagnostics + flywheel)	—	deferred	deferred	deferred	deferred
agents_* (registry)	—	deferred	—	deferred	deferred
maintenance_* (operator)	—	—	—	—	eager
memory:// resources	—	deferred	deferred	deferred	deferred
prompts (workflow)	—	deferred	deferred	deferred	deferred

Figure 12.1 — Profile × tool-family visibility. `agent_brain` is the narrowest surface (the five-method facade); `coder` adds memory, KG, and feedback; `operator` is the only profile that sees the maintenance tools.

12.1 brain_* — agent facade

The narrowest surface, mapped one-to-one onto the `AgentBrain` Python facade. The first six are the daily-driver subset most agents see; `brain_audit_consumers` and `brain_export` are added for compliance and migration paths.

Tool	Required args	Purpose
brain_remember	fact	Save a memory. Optional tier (default <i>procedural</i>), <i>share=True</i> or <i>share_with</i> for Hive.
brain_recall	query	Ranked memories with composite score. Optional <i>max_results</i> , <i>scope</i> filter.
brain_forget	key	Archive (reversible). Not the same as <i>memory_delete</i> (permanent).
brain_learn_success	description	Record a successful task outcome. Lifts confidence on touched memories.
brain_learn_failure	description	Record a failed approach. Drops confidence; surfaces failed-approaches on related recalls.

Tool	Required args	Purpose
brain_status	—	Identity, group membership, store stats, Hive connectivity.
brain_audit_consumer s	—	List every project / agent that has read this brain. Compliance + data-residency audit.
brain_export	—	Whole-brain JSON export for migration, backup, or relay handoff.

Example — save and recall (MCP JSON envelopes)

Every MCP tool call looks like a JSON envelope with *tool* and *arguments*. A typical save with cross-agent sharing:

```
{ "tool": "brain_remember", "arguments": { "fact": "All services must use structured JSON logging", "tier": "architectural", "share_with": "hive" } }
```

And the matching recall:

```
{ "tool": "brain_recall", "arguments": { "query": "logging conventions", "max_results": 5 } }
```

The recall returns a list of result objects with *key*, *value*, *tier*, *score*, *agent_scope*, *tags*, *confidence*, and, when present, *failed_approaches*. An empty list does not mean the store is empty — it means the diagnostic gate (circuit-breaker state, below-threshold floor, scope filter, RAG safety block) suppressed the hits. Operators call the deeper *memory_recall* tool to get back a *RecallResult* with an explicit *empty_reason* code.

12.2 memory_* — full CRUD and search

The full read/write surface — 25 tools covering `save/get/list/search/delete`, bulk operations, tagging, supersession, relations, session indexing, audit queries, export/import, and GC/consolidation configuration. Most operate on individual keys; `memory_save_many` and `memory_reinforce_many` exist for bulk paths. `memory_supersede` creates a new version that structurally points back to the old one (via `superseded_by`), letting the recall path silently prefer the new version while preserving the audit chain.

Sub-family	Key tools	When to reach for it
CRUD	<code>memory_save</code> , <code>memory_get</code> , <code>memory_list</code> , <code>memory_delete</code>	Direct key-addressed operations. Use when you already know the key.
Search	<code>memory_search</code> , <code>memory_recall</code> , <code>memory_recall_many</code>	When you need an explicit <code>empty_reason</code> code or batch recall.
Reinforcement	<code>memory_reinforce</code> , <code>memory_reinforce_many</code> , <code>memory_capture</code> , <code>memory_ingest</code>	Bank a positive recall signal; capture facts from an agent response.
Versioning	<code>memory_supersede</code> , <code>memory_history</code> , <code>memory_update_tags</code>	Atomic replace-with-rationale; tag changes without rewriting the body.
Relations	<code>memory_relations</code> , <code>memory_relations_get_batch</code> , <code>memory_query_relations</code> , <code>memory_find_related</code>	Project-local lightweight relation graph. BFS by hop count.
Discovery	<code>memory_list_tags</code> , <code>memory_list_groups</code> , <code>memory_entries_by_tag</code>	Inventory the tag landscape before inventing a new tag.
Session	<code>memory_index_session</code> , <code>memory_search_sessions</code> , <code>tapps_brain_session_end</code>	Native session-chunk indexing; end-of-session episodic write.
Audit	<code>memory_audit</code> , <code>memory_export</code> , <code>memory_import</code>	Compliance queries; backup; migration between deployments.
Config	<code>memory_gc_config</code> , <code>memory_gc_config_set</code> , <code>memory_consolidation_config</code> , <code>memory_consolidation_config_set</code>	Profile-level GC and consolidation tuning.

12.3 kg_* — Knowledge Graph

The Knowledge Graph surface (EPIC-076) lets agents reason about *relations between things*: 'what other workflows touched this database?', 'how does module X relate to service Y?'. Entities are upserted on (`brain_id`, `entity_type`, `canonical_name`). Edges carry a confidence; edges without supporting evidence are **capped at 0.4** (ADR-012) so the recall path can never rank an unsupported claim ahead of a supported one.

Tool	Purpose
brain_record_event	Atomic write of one ExperienceEvent + optional memory + entities + edges + evidence in a single Postgres transaction. Rollback is all-or-nothing across all five tables.
brain_record_events_batch	Same, but for many events. Per-event TX (independent rollback) or single-TX (all-or-nothing) selectable at the call site.
brain_get_neighbors	1-hop or 2-hop neighbourhood around one or more entities.
brain_explain_connection	Shortest path (≤ 3 hops) between two entities. Returns the path edges and their evidence rows.
brain_record_feedback	edge_helpful / edge_misleading signal for KG edges. Drives FSRS-like edge stability updates.

Example — atomic KG write

```
{  "tool":  "brain_record_event",  "arguments":  {    "event_type":  "workflow_completed",    "utility_score":  0.85,    "payload_json":  "{\"workflow\":\"plan_and_implement\",\"steps\":4}\",    \"entities_json\":  \"[{\"entity_type\":\"workflow\",\"canonical_name\":\"plan_and_implement\",\"confidence\":0.9}]\",    \"memory_key\":  \"recall-worked-well\",    \"memory_value\":  \"Hybrid recall improved precision on architecture queries.\",    \"memory_tier\":  \"pattern\"  }  }
```

12.4 hive_* — cross-agent sharing

Hive is the cross-agent layer inside one project. Six tools manage the shared namespace surface: *hive_status* (introspection), *hive_search* (read), *hive_propagate* (single-key publish from local to Hive), *hive_push* (bulk publish with filters), and a *hive_write_revision* / *hive_wait_write* pair backed by Postgres *LISTEN/NOTIFY* for change-feed clients. The revision is a monotonic counter on the *hive_write_notify* singleton row, incremented on every Hive insert.

12.5 feedback_* and diagnostics — the quality loop

Thirteen tools driving the deterministic quality loop: *feedback_rate* (per-recall useful/not), *feedback_gap* (signals 'the brain should have known this but didn't'), *feedback_issue* (flags an entry as wrong), *diagnostics_report* (composite scorecard with EWMA anomaly state), *diagnostics_history* (persisted snapshots), and the *flywheel_** tools that translate feedback into confidence updates and a markdown quality report. None of these tools call an LLM in the request path. *flywheel_evaluate* can optionally consult an LLM judge in **offline** mode for deeper scoring; this is operator-gated.

12.6 agents_* — the agent registry

Four tools: *agent_register*, *agent_create*, *agent_list*, *agent_delete*. The registry stores agent identity, profile, declared skills, project root, group memberships, and a heartbeat timestamp in the *agent_registry* Postgres table. Hive members can introspect peers via *agent_list*; expert discovery uses *agents_for_domain(domain_name)*.

12.7 maintenance_* — operator only

Thirteen tools gated behind the **separate** *tapps-brain-operator-mcp* server on port 8090 with its own admin token. The standard agent server **never** exposes these, even if *TAPPS_BRAIN_OPERATOR_TOOLS=1* is set in the agent's environment. Operator tools are: *maintenance_gc*, *maintenance Consolidate*, *maintenance_stale*, *tapps_brain_health*, *memory_gc_config[_set]*, *memory Consolidation_config[_set]*, *memory_export*, *memory_import*, *tapps_brain_relay_export*, *flywheel_evaluate*, *flywheel_hive_feedback*.

12.8 memory:// resources and workflow prompts

Eight read-only resources for introspection: *memory://stats*, *memory://agent-contract*, *memory://health*, *memory://entries/{key}*, *memory://metrics*, *memory://feedback*, *memory://diagnostics*, *memory://report*. Three workflow prompts ship: *recall(topic)*, *store_summary()*, *remember(fact)* — used by MCP clients that surface user-invoked templates.

13. The Skills Layer

20 .claude skills + 11 specialist sub-agents — the routing layer between user intent and tools.

Skills are **tool compositions**. A skill is a short markdown playbook with trigger phrases, mandatory tool ordering, and post-conditions. The harness loads the skill on demand; the agent reads it and follows the routing. Skills are how the system enforces 'always call `validate_changed` before `checklist`' without bloating the system prompt.

13.1 Core memory skills

Skill	Trigger	What it does
tapps-brain	"recall prior context", "what did we decide", "Hive"	Thin wrapper that routes to the deployed brain HTTP service via MCP. Daily-driver subset of 8 tools (brain_recall, brain_remember, brain_status, brain_get_neighbors, brain_explain_connection, memory_search, memory_find_related, hive_search).
tapps-memory	"save to memory", "consolidate", "federation"	42-action memory surface routed through <i>tapps_memory</i> so the call is filtered by profile, tier rules, content safety, and feedback-flywheel auto-emission. Never call <i>save_issue</i> /raw primitives directly.
tapps-init	"bootstrap TappsMCP", "new project setup"	One-shot scaffolder. Writes AGENTS.md / TECH_STACK.md / platform rules / hooks / agents / skills / .mcp.json. Returns <i>content_return: true</i> when the server cannot write directly (Docker mode).
tapps-research	"research library", "best practices", "API docs"	Routes to <i>tapps_lookup_docs(library, topic)</i> . The lookup is Context7-backed and locally cached; repeat calls are near-zero cost.
tapps-finish-task	"end-of-task", "declare work complete"	Mandatory 3-step pipeline: (1) <i>validate_changed</i> with explicit <i>file_paths</i> , (2) <i>checklist</i> with the appropriate <i>task_type</i> , (3) optional memory save if the session learned anything architectural. Fails closed: any step failing aborts the pipeline.
tapps-gate	"quality gate", "pass/fail gate"	Single-call wrapper on <i>tapps_quality_gate</i> . Surfaces blocking issues; security floor 50/100 is non-overridable.
tapps-security	"security audit", "CVE check"	Comprehensive security audit including dependency CVE scan.
tapps-validate	"validate before merge"	Pre-completion validation across changed files.
tapps-score, tapps-report	"score this file", "quality report"	Per-file scoring and aggregate project reporting.

Skill	Trigger	What it does
tapps-engagement	"set engagement level"	Switches the recall/inject engagement (high/medium/low — controls the token budget for memory injection).
tapps-tool-reference, tapps-apply-files	lookup, side-effect apply	Reference docs / apply file ops returned by content-mode tools.

13.2 Linear and release skills (one-of-many integrations)

linear-issue, *linear-read*, *linear-release-update* route Linear writes through the docs-mcp validator before they reach the plugin. These are illustrative — they show how a skill *mandates* a generator-then-validator-then-save sequence so that no agent can shortcut into a raw plugin call. The same shape is used by the release-update skill: *tapps_release_update* → check *agent_ready=true* → *save_document* → *tapps_linear_snapshot_invalidate*.

13.3 Specialist sub-agents — Ralph loop and TAPPS review

Eleven specialist sub-agents back the autonomous workflows. **Seven *ralph*-* sub-agents** drive the autonomous-development loop: *ralph* (the driver), *ralph-architect* (heavy reasoning), *ralph-coordinator* (task-brief retrieval from tapps-brain), *ralph-explorer* (read-only codebase search), *ralph-tester* and *ralph-bg-tester* (synchronous and background test runs), and *ralph-reviewer* (pre-commit review). **Four *tapps*-* sub-agents** back the quality pipeline: *tapps-researcher* (library and best-practice lookups), *tapps-review-fixer* (combined score-then-fix worker for parallel review pipelines), *tapps-reviewer* (read-only quality enforcement after Python edits), and *tapps-validator* (pre-completion validation). The matching skills — *ralph-loop*, *ralph-workflow*, *ralph-research*, *tapps-review-pipeline* — sit one layer up and orchestrate these sub-agents. None of the sub-agents talk to memory directly; they all talk through *tapps-brain* and *tapps-memory*.

14. Hooks and System Prompts

The harness-side levers that make the agent reliably call the brain.

Skills route the agent *when it gets there*. Hooks make sure it gets there. The harness fires events — *SessionStart*, *SessionEnd*, *PreCompact*, *PreToolUse*, *PostToolUse*, *Stop* — and shell scripts in `.claude/hooks/` respond by emitting *system-reminder* text that the agent reads on the next turn.

14.1 The SessionStart recall primer

The single most important hook in a tapps-brain deployment is the *SessionStart* recall primer. It fires on every fresh launch and on `claude --continue`. Its content is short, deterministic, and operator-readable:

Hook script (`.claude/hooks/tapps-brain-session-start.sh`):

```
#!/usr/bin/env bash
INPUT=$(cat)
BRANCH=$(git -C "${CLAUDE_PROJECT_DIR:-.}" rev-parse --abbrev-ref HEAD 2>/dev/null || echo "")
echo "Before answering the user's first message, call brain_recall via the tapps-brain MCP server."
echo "Query: the user's opening topic (architecture, a specific module, a recent epic, or the current branch '${BRANCH}')."
echo "This primes cross-session memory. Skip the recall only if the user asked a trivial question (e.g. 'what time is it')."
exit 0
```

Two observations: **(1)** the hook does not call the brain itself — it tells the agent to call the brain, leveraging the already-open MCP session. **(2)** the matcher is *startup|resume*, so it fires on both fresh launches and continued sessions. The cost is negligible (~50–200 ms for the MCP round-trip on the next turn), and it eliminates the most common integration failure mode: 'Claude forgot to recall on turn 1'.

14.2 Other hooks in the standard configuration

PreToolUse plugin save	on raw	Blocks raw Linear <i>save_issue</i> calls if a recent <i>docs_validate_linear_issue</i> sentinel is missing. Forces the <i>linear-issue</i> skill flow.
PostToolUse list_issues	on	Auto-populates the local Linear snapshot cache from the response, so the agent's next read can be cache-served.
PreToolUse list_issues	on	Cache-gate. Rejects (or warns) when no recent <i>tapps_linear_snapshot_get</i> sentinel exists. Cross-project reads pass through with a <i>cross_project</i> tag in the violation log.
SessionEnd / Stop		Optional <i>memory_ingest</i> with the transcript summary. Deferred in the default config — operators turn it on once they trust their session-capture pipeline.
PreCompact		Backs up scoring context before the harness compresses the conversation. Not duplicated by the <i>SessionStart</i> primer because the primer re-injects memories into the post-compact context automatically.

14.3 Why not put 'always call brain_recall' in CLAUDE.md?

CLAUDE.md instructions are advisory: a fresh model can still forget them. Hooks are *imperative*: the harness emits the reminder exactly when the matching event fires, and the agent reads it on the very next turn. The two layers compose — CLAUDE.md explains *when* to call the brain (the behavioral rules); the hook makes sure the call *actually happens* on the turn that matters most (the first one).

15. Profiles — Tier and Decay Shapes

Six built-in profiles, the math behind their decay curves, and how to choose.

A profile is a YAML document that defines the agent's **tier stack**, **decay parameters**, **scoring weights**, **importance-tag multipliers**, **promotion rules**, and **Hive propagation policy**. tapps-brain ships six built-in profiles, each tuned to a domain:

repo-brain	Code repositories and AI coding assistants. Power-law decay ($\beta \approx 3.29$, FSRs-anchored).
customer-support	Support agents. Exponential decay. Frequency-upweighted scoring (recurring issues bubble up).
home-automation	IoT and home automation. Mixed decay (very slow on household-profile, fast on transient). Safety-tagged memories get a 3× importance multiplier.
personal-assistant	Personal memory. Long household tier; recency-leaning.
project-management	PMO / programme management. Pattern-dominant.
research-knowledge	Research notes. Long architectural tier; confidence-emphasised scoring.

15.1 Decay curves — exponential vs power-law

tapps-brain supports two decay models per tier. Exponential decay (the Ebbinghaus baseline) drops weight by half every half-life period: $\phi(t) = \exp(-\ln(2) \cdot t / h)$. Power-law decay (the FSRs and Wixted finding) preserves the $R(h) = 0.5$ median-retention property but produces a fatter tail, which is what code-related knowledge actually behaves like: yesterday's architectural decision is still relevant six months later, where the exponential curve has it nearly forgotten. The repo-brain profile uses power-law decay on all four tiers, with the canonical FSRs exponent $\beta \approx 3.29$ (derived from $k = 81/19$):

$$R(t) = (1 + t / h)^{-\beta} \text{ with } \beta = \ln 2 / \ln (1 + 1/k) \text{ (FSRS canonical } \beta \approx 3.29 \text{ for } k = 81/19)$$

Tier-aware memory decay (repo-brain profile)

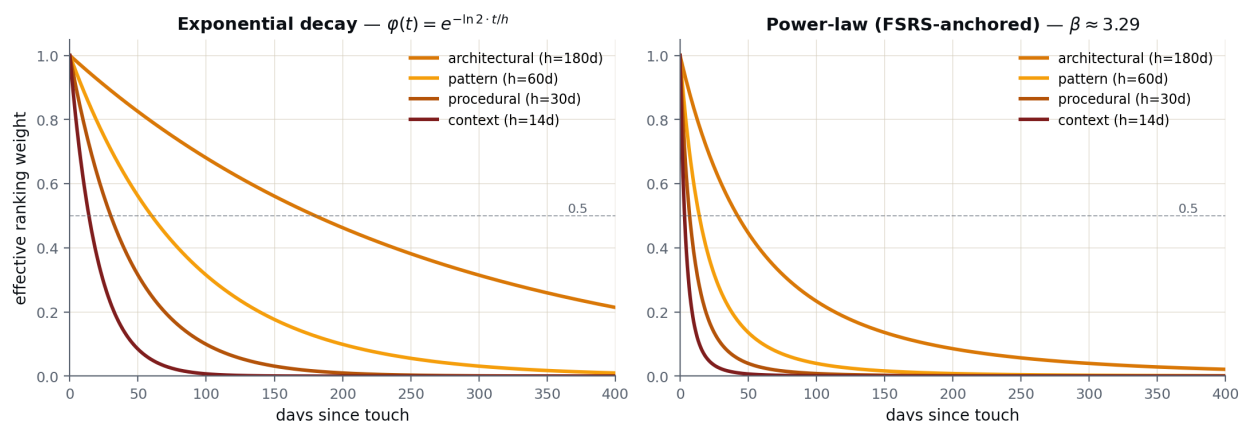
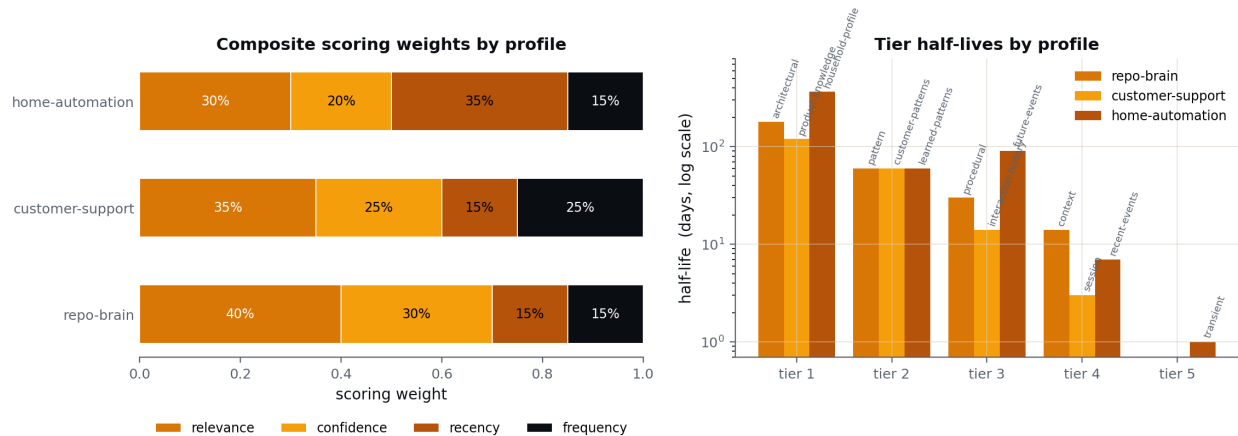


Figure 15.1 — Exponential vs power-law decay for the four default repo-brain tiers. Both curves pass through $\phi(h) = 0.5$; the right-hand power-law tail is materially fatter — architectural decisions remain influential well past their nominal half-life.

15.2 Profile comparison — three contrasting profiles

The three profiles below were chosen because they sit at the corners of the design space. **repo-brain** is relevance-first with long-tail decay. **customer-support** upweights frequency (recurring tickets) and uses exponential decay (tickets *do* become irrelevant on a wall-clock schedule). **home-automation** upweights *recency* (the state of the house literally changes minute to minute) and tags safety-related memories with a 3× multiplier so they never lose their place at the top of recall.



15.3 Importance-tag multipliers

Beyond tier-level decay, every profile assigns a multiplier to specific tags. **repo-brain** uses *critical*: 2.0, *security*: 1.5. **customer-support** uses *policy*: 2.0, *pricing*: 1.5, *known-issue*: 1.5, *escalation*: 1.5. **home-automation** uses *safety*: 3.0, *allergy*: 3.0, *medical*: 3.0. These multipliers are applied *after* composite scoring, so a safety-tagged entry that would otherwise rank in the middle of the pack lands at the top — the right behaviour for an IoT agent making decisions about a house with allergic residents.

15.4 Promotion and demotion

Profiles also declare tier **promotion** rules: when an entry crosses a configured threshold (typically *min_access_count* ≥ 10, *min_age_days* ≥ 30, *min_confidence* ≥ 0.7), the entry is promoted to the longer-half-life tier above it. Demotion runs the opposite direction on confidence collapse. This is what gives the corpus its self-organising property: an entry that proves repeatedly useful migrates toward longer durability, and an entry that is contradicted drifts toward the short-lived context tier.

16. Call Tree — Recall

The hot-path function-by-function: agent query → ranked memories.

Recall is the most-called operation; understanding its call tree is the difference between debugging an outage in minutes versus hours. The entry is the MCP tool body in `services/memory_service.py`; the deepest synchronous frame is the Postgres backend's parameterised query.

Hybrid recall pipeline — lexical + dense + (optional rerank), fused by RRF before lazy decay and token-budget injection

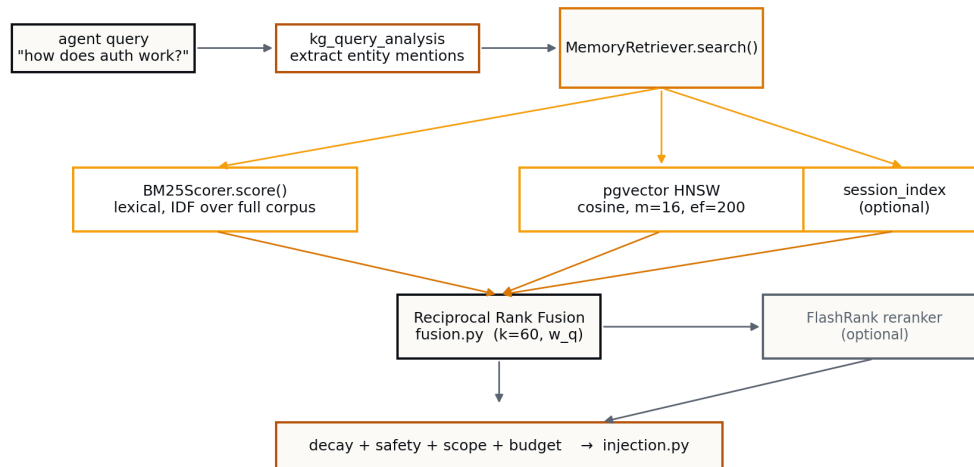


Figure 16.1 — The hybrid recall pipeline. Three retrieval channels (BM25, pgvector, session-index) merged by Reciprocal Rank Fusion, then optional FlashRank rerank, then decay / safety / scope / budget filters, then injection formatting.

16.1 Function-level call tree

Each indent is one frame deeper. *file:line* references are anchored to the v3.20.x source tree:

```

brain_recall services/memory_service.py:175
■ otel_tracer.start_mcp_tool_span("brain_recall")
■ store.recall(query, ...) store.py:2591
  ■ RecallOrchestrator.recall(message) recall.py:153
    ■ kg_query_analysis.analyze(query) — extract entity mentions
    ■ inject_memories(message, store, ...) injection.py:222
    ■ MemoryRetriever.search(query) retrieval.py:100+
    ■ store.search → PostgresPrivateBackend.search postgres_private.py:174
    ■ SQL: WHERE tsvector @@ plainto_tsquery('english', $1)
    ■ BM25Scorer.score(tokens, corpus) bm25.py
    ■ Okapi BM25 with full-corpus IDF
    ■ pgvector ANN postgres_private.py search_vectors
    ■ SQL: ORDER BY embedding <=> $1 LIMIT pool
    ■ fusion.reciprocal_rank_fusion(bm25, vector) fusion.py:42
    ■ hybrid_rrf_weights_for_query(query) → per-query weights
    ■ FlashRankReranker.rerank(top_20) reranker.py:79 (optional)
    ■ decay.calculate_decayed_confidence(entry, now) decay.py
    ■ safety.check_content_safety(value) safety.py
    ■ apply scope / tier / branch / group filters injection.py
    ■ format_memory_section(memories, token_budget)
    ■ hive_store.search(message, [universal, profile, groups]) postgres_hive.py
    ■ merge with hive_recall_weight (default 0.8)
    ■ apply graph boost (relations graph, max_hops=2)
    ■ recall_quality_buffer.record_sample(top_score, age_days, count)
  
```

16.2 Worked example

Agent calls *brain_recall*(*query*="how does auth work?", *max_results*=5):

1. **tools/call** arrives at *tool_filter* with profile *coder*. *brain_recall* is in the eager set; allowed.
2. *services.brain_recall* opens an OTel span *tapps_brain.recall* with *gen_ai.operation.name=recall* (GenAI semconv).
3. *store.recall* lazy-initialises the *RecallOrchestrator* (cached for the life of the process).
4. *kg_query_analysis* extracts the entity mention 'auth' and calls *batch_resolve_entities* against Postgres KG. Returns *QueryAnalysis(entities=['auth-service'])*.
5. Two retrieval channels fire in parallel inside *MemoryRetriever.search*: **(a)** the lexical channel runs *plainto_tsquery*('english', 'how does auth work?') on the *tsvector* column, gets ~35 candidate rows, then re-ranks them with BM25 against the full-corpus IDF. **(b)** the dense channel computes the SentenceTransformer embedding of the query and runs an HNSW kNN query (*m*=16, *ef*=200, cosine). Each channel returns ~40 candidates with ranks.
6. *fusion.reciprocal_rank_fusion* merges the two lists with *k*=60. *hybrid_rrf_weights_for_query* tags this query as *balanced* (no strong code-symbol or quoted-pharse signal); *w_bm* = *w_vec* = 1.0.
7. Optional FlashRank rerank runs over the top 20. Score reshuffles 3 of the top 5.
8. *decay.calculate_decayed_confidence* applies power-law decay ($\beta \approx 3.29$) per entry, using its tier's half-life. Two entries fall below *_MIN_SCORE*.
9. *safety.check_content_safety* scans each candidate for prompt-injection patterns. None matched.
10. *inject_memories* packs the remaining candidates into a token budget of 3000 tokens at *engagement_level=high*, with the formatted section getting 60% of the budget. ~36 memories fit.
11. *hive_store.search* runs the same query in the Hive across namespaces [*universal*, *repo-brain*, *team-backend*] (this agent's memberships). 8 additional hits not present locally; weight 0.8× applied.
12. Graph boost: for each memory, *find_related* walks the relations graph up to 2 hops. Two connected entries get a +0.05 boost.
13. *recall_quality_buffer* records sample {*top_score*=0.91, *oldest_age*=45 d, *memory_count*=42}.
14. Return *RecallResult* with *memory_section* (formatted markdown ready for injection into the system prompt), *memories* list, *token_count*, *recall_time_ms*, *hive_memory_count*=8, and *recall_diagnostics*.

17. Call Tree — Save / Remember

The write pipeline: safety, conflict, persist, propagate, consolidate.

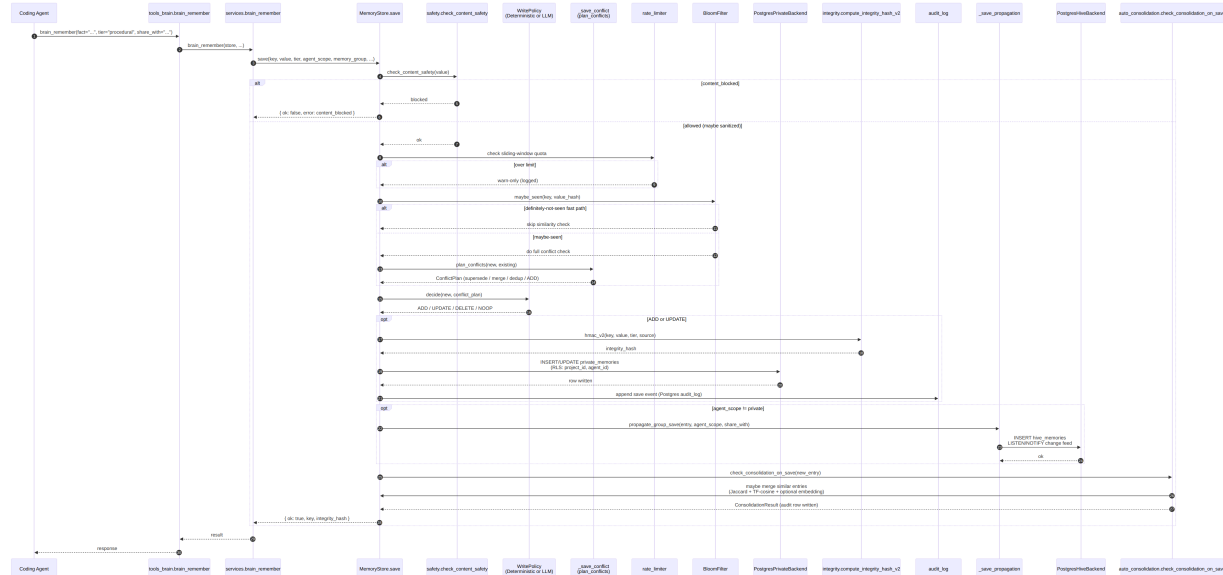


Figure 17.1 — End-to-end save sequence. The Bloom-filter fast path skips full conflict planning on definitely-not-seen writes.

17.1 Function-level call tree

```

brain_remember services/memory_service.py:25
■ ■ ■ resolve agent_scope / share_with (TAP-989)
■ ■ ■ optional supersession detection memory_service.py:92
■ ■ ■ store.save(key, value, tier, agent_scope, ...) store.py:889
    ■ ■ ■ Phase 1: validate scope and group _save_pipeline.py
    ■ ■ ■ Phase 2: safety.check_content_safety(value) safety.py
    ■ ■ ■ Phase 3: write_policy.decide() write_policy.py
        ■ ■ ■ Deterministic ADD / UPDATE / NOOP
    ■ ■ ■ Phase 4: dedup fast path
        ■ ■ ■ BloomFilter.maybe_seen(key, value_hash) bloom.py
    ■ ■ ■ Phase 5: conflict planning _save_conflict.py:50
        ■ ■ ■ contradictions.detect_save_conflicts() contradictions.py
        ■ ■ ■ plan_conflicts() → ConflictPlan(SUPERSEDE / MERGE / DEDUP / ADD)
    ■ ■ ■ Phase 6: build entry
        ■ ■ ■ _construct_memory_entry() store.py:1352
        ■ ■ ■ integrity.compute_integrity_hash_v2() integrity.py
        ■ ■ ■ embeddings.embed(value) embeddings.py (if provider enabled)
    ■ ■ ■ Phase 7: persist
        ■ ■ ■ PostgresPrivateBackend.save(entry) postgres_private.py:174
        ■ ■ ■ INSERT/UPDATE private_memories
        ■ ■ ■ NOTIFY taps_memory_change
        ■ ■ ■ postgres_private.append_audit() postgres_private.py:530
    ■ ■ ■ Phase 8: propagation
        ■ ■ ■ _propagate_to_hive() store.py:1641
        ■ ■ ■ PropagationEngine.propagate() → PostgresHiveBackend.save()
        ■ ■ ■ propagate_group_save() _save_propagation.py:29
        ■ ■ ■ publish_to_experts() _save_propagation.py:74 (private + arch/pattern + expert)
    ■ ■ ■ Phase 9: relations extract postgres_private.py:451
    ■ ■ ■ Phase 10: auto_consolidation.check_consolidation_on_save()
  
```

17.2 Phase-by-phase latency budget

The save pipeline is the only end-to-end write critical path for an agent; knowing where the time goes is essential when sizing the connection pool and the bloom filter. Phases are independently instrumented (`store.save.phase.*` histograms); the shape below is the typical production curve.

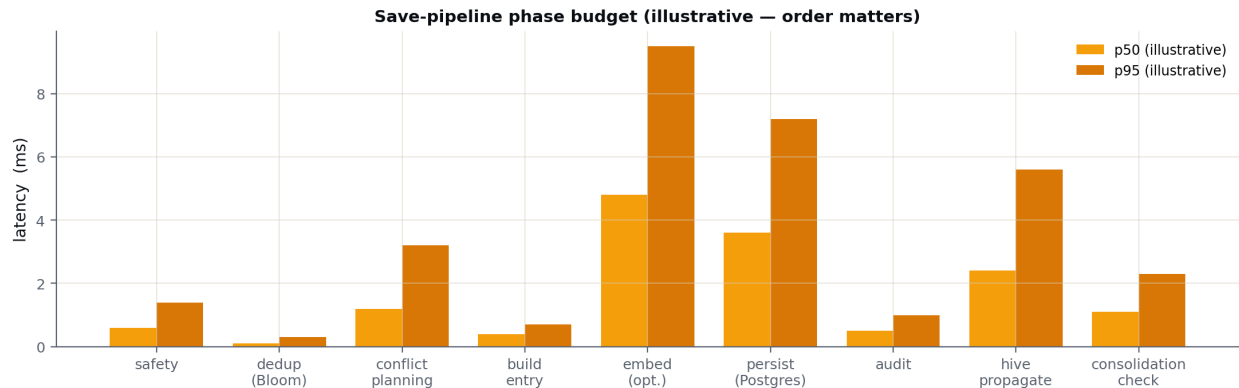


Figure 17.2 — Save-pipeline phase budget (illustrative). The embed step dominates when the SentenceTransformer provider is on; persist is the second-largest term on a hot cache, and the largest on a cold one.

17.3 Worked example

Agent calls `brain_remember(fact="auth uses JWT with RS256 signing", tier="architectural", share_with="hive")`:

1. Resolved scope: `agent_scope="hive"` (TAP-989 — explicit `share_with` wins over `share=True`).
2. Generated key: `auth-uses-jwt-7a3b2c9d1e5f4a2b` (content hash slug).
3. `safety.check_content_safety` scans for prompt-injection patterns. Clean. Continue.
4. `write_policy.decide`: deterministic mode → `ADD`.
5. `BloomFilter.maybe_seen`: definitely-not-seen; skip full conflict scan.
6. Build `MemoryEntry` with `tier=architectural`, `source=agent`, `agent_scope=hive`, `created_at=now`. Compute `integrity_hash` with HMAC v2. Compute the 384-dim embedding via SentenceTransformer.
7. `postgres_private.save` writes the row and `NOTIFYs` `tapps_memory_change`. `append_audit` writes `{event_type: save, key: ..., tier: architectural, ...}`.
8. `PropagationEngine.propagate` resolves namespaces for `agent_scope=hive`: `[universal, repo-brain, team-backend, platform]`. `PostgresHiveBackend.save` writes the row to each, increments `hive_write_notify.revision`, and emits `NOTIFY hive_memories_changed`.
9. `extract_relations` identifies the triple (`JWT`, `signs_with`, `RS256`) and writes it to `relations` table.
10. `auto_consolidation.check_consolidation_on_save` scans for entries > 0.7 similarity. None match the threshold; merge skipped.
11. Return `{saved: true, key: "auth-uses-jwt-...", supersession_candidate: null}`.

18. Agent-to-Agent Flows

Scopes, namespaces, the LISTEN/NOTIFY change feed, and the agent registry.

Multi-agent operation rests on four primitives: the **agent_scope** string (carried on every save), the **Hive namespace** the propagation engine writes to, the **LISTEN/NOTIFY change feed** that wakes interested subscribers, and the **agent registry** that lets peers introspect each other's capabilities.

18.1 Scope semantics

agent_scope	Destination namespace	Read access
private	(none — private_memories only)	Only the saving agent.
domain	domain:<profile>	Agents with the same profile (e.g. all <i>repo-brain</i> agents).
hive	universal	All agents in the project.
group:<name>	group:<name>	Only registered members of the group. Writes from non-members are rejected at the propagation layer.

There is one additional *implicit* path: an agent that saves *agent_scope=private* with tier in *{architectural, pattern}* and has declared *expert_domains* will be auto-published to *universal* with a *expert:<domain>* tag. This is the 'expert auto-publish' pattern: a frontend specialist's private decision about a CSS pattern is automatically broadcast to the project's universal namespace, tagged so peers can filter on it.

18.2 Hive change feed (LISTEN/NOTIFY)

Every Hive insert increments a monotonic *revision* counter on the single-row *hive_write_notify* table and emits a Postgres *NOTIFY hive_memories_changed* pulse. Clients can either:

- **Subscribe** via the Hive backend's *LISTEN* path, blocking on *select.select()* over the connection's file descriptor until either the next pulse arrives or the timeout expires.
- **Poll** via the *hive_wait_write(since_revision, timeout_sec)* MCP tool, which falls back to a polling loop when the connection is pool-borrowed and cannot hold a long-lived *LISTEN*.

The return shape is the same in both paths: *{revision: int, updated_at: ISO-8601, changed: bool, timed_out: bool}*. Clients use *changed=true* to invalidate caches, refresh local copies, or trigger their own work loops.

18.3 Agent registry

PostgresAgentRegistry is the system of record for agent identity and capability metadata. Every registered agent has a row in *agent_registry* with: **id** (slug), **name**, **profile**, **skills** (JSONB list), **project_root**, **groups** (JSONB list), **registered_at**, **last_seen_at**. The four MCP tools (*agent_register*, *agent_create*, *agent_list*, *agent_delete*) wrap the CRUD operations; *agents_for_domain(domain_name)* is the discovery primitive the propagation engine uses when fanning out expert-auto-publishes.

18.4 Fleet topology — what an operator sees

Agent fleet topology — N project containers, one brain sidecar, one Postgres (RLS-enforced per-project isolation)

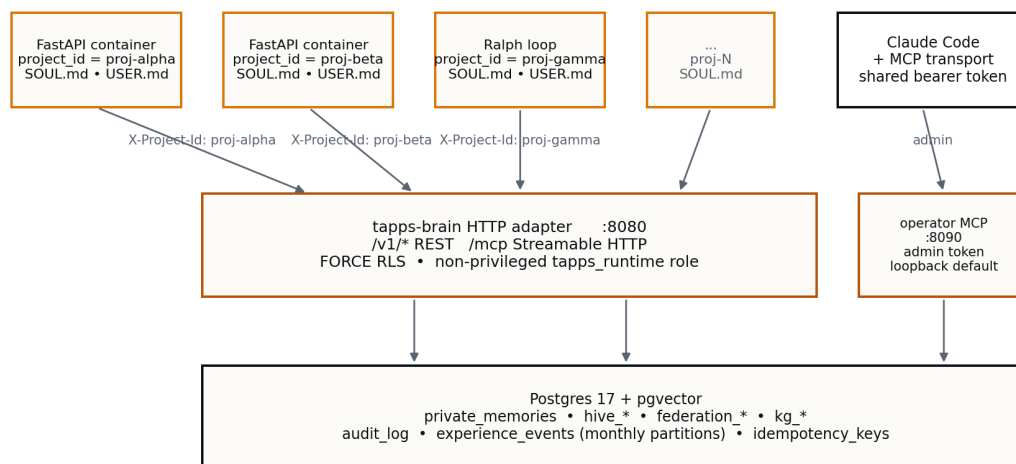


Figure 18.1 — Reference fleet: N project containers (each with their own *SOUL.md* / *USER.md* and a project-id header on every brain request), one tapps-brain sidecar serving */mcp* and */v1/** on :8080 with FORCE RLS and the non-privileged *tapps_runtime* role, one operator MCP on :8090 (admin token, loopback by default), one Postgres with the canonical durable tables.

18.5 Federation — cross-project sharing

Federation is the cross-project layer. It is **not** automatic — projects explicitly publish and subscribe through *PostgresFederationBackend*. Published rows land in *federated_memories* with the publisher's *project_id* and (optionally) their local *memory_group*. Subscribers *pull* on demand via the federation hub — there is no background replication. This is a conscious choice: federation is for amortising organisation-wide knowledge across many projects, not for approximating real-time multi-master replication.

End-to-end propagation example. Agent A (project P, profile *repo-brain*, group memberships [*backend*, *platform*], expert_domains [*auth*, *jwt*]) saves *brain_remember(fact=..., tier="architectural", agent_scope="private")*. The save lands in P's *private_memories*. Because the tier is *architectural* and the agent has *expert_domains*, the expert auto-publish path also writes the entry to *hive_memories* in namespace *universal* with tag *expert:auth*. Agent B in the same project sees the entry on its next recall. Some weeks later, project P's operator runs federation publish on the same key; the entry lands in *federated_memories*. Project Q subscribes to project P and runs a sync; the entry now appears in Q's *private_memories* with *source=federation* attribution.

19. Sequence Diagrams — Full Reference

The four canonical flows in their high-resolution Mermaid form.

These are the canonical sequence diagrams generated from the source tree. They are reproduced here in case the chapter-by-chapter walks are not enough; each diagram is a complete trace from agent call to backend write.

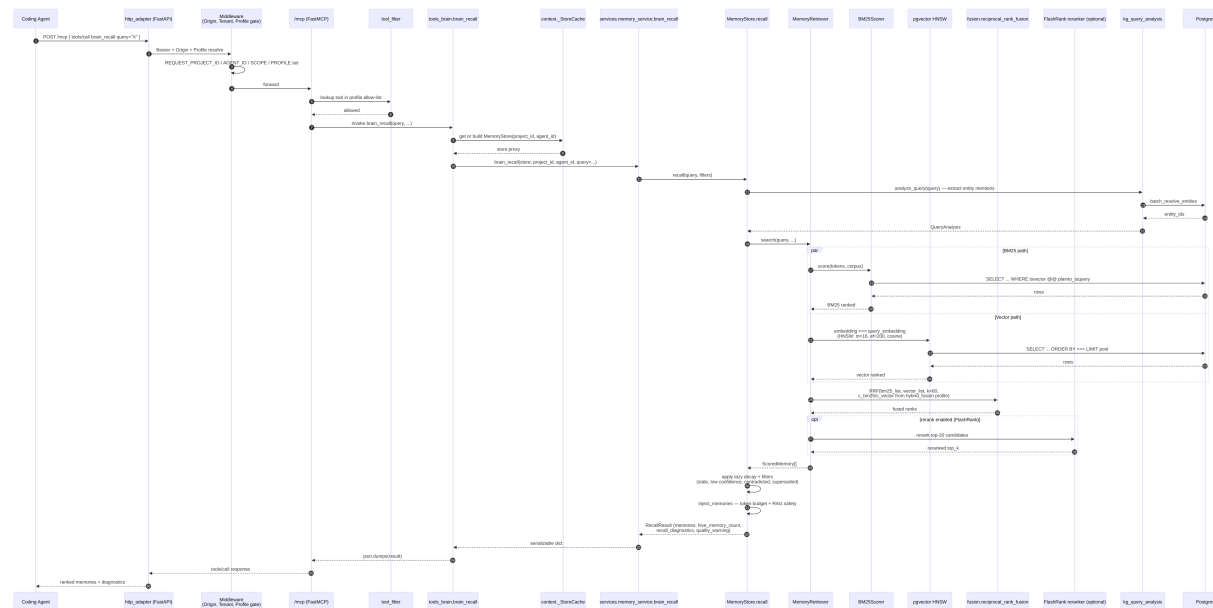


Figure 19.1 — Full *brain_recall* sequence. Parallel BM25 + pgvector channels, RRF, optional rerank, lazy decay, scope filter, token budget.

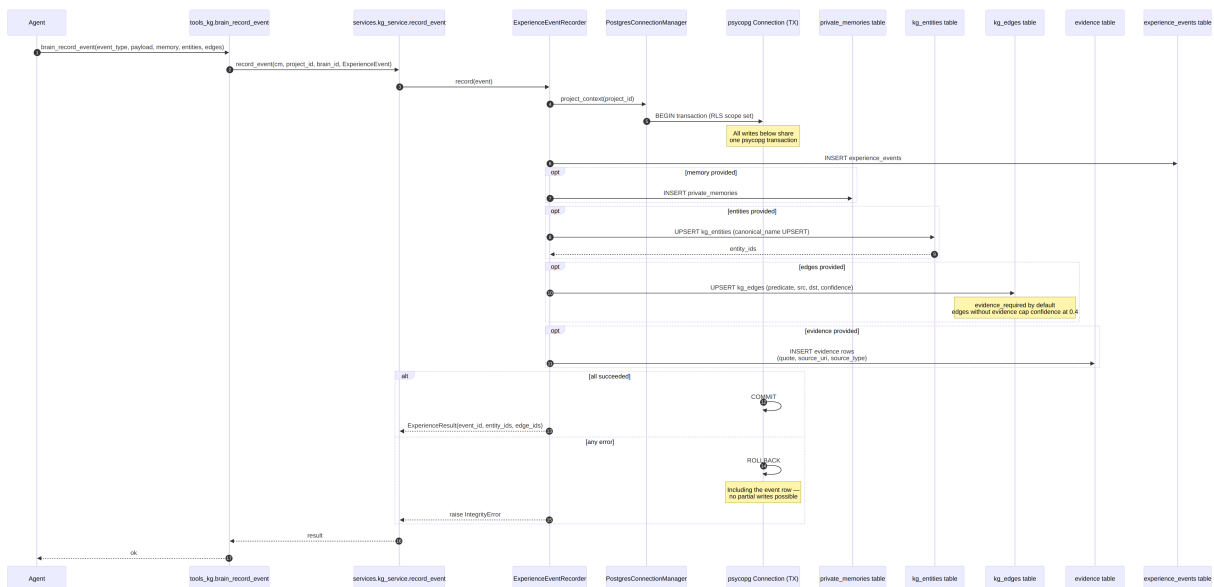


Figure 19.2 — Full *brain_record_event* sequence — the atomic Knowledge-Graph write. Five tables (*experience_events*, *private_memories*, *kg_entities*, *kg_edges*, *evidence*) under a single Postgres transaction.

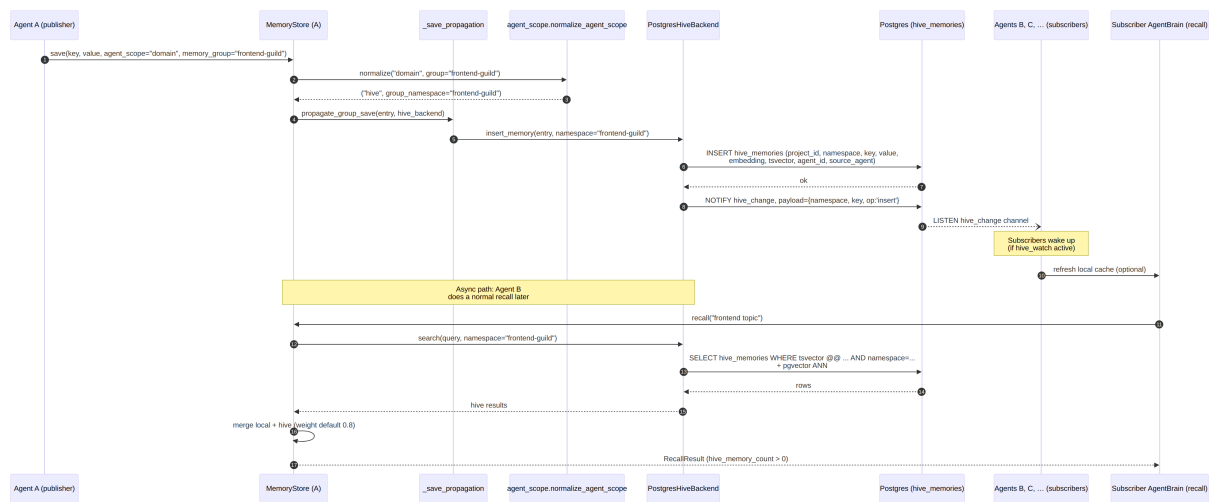


Figure 19.3 — Full Hive propagation sequence. The publish side is one Postgres insert plus a NOTIFY; the consume side reads through normal namespace-aware recall.

20. Mathematical Foundations

The deterministic formulas behind every ranking, every decay, every merge.

20.1 Okapi BM25

Lexical relevance is scored with Okapi BM25, the de-facto standard for keyword retrieval (Robertson and Zaragoza, 2009). For a query Q tokenised into terms $q_1, \dots, q_n$ and a candidate document D with length $|D|$, BM25 sums per-term contributions:

$$\text{BM25}(D, Q) = \sum_i \text{IDF}(q_i) \cdot [f(q_i, D) \cdot (k_1 + 1)] / [f(q_i, D) + k_1 \cdot (1 - b + b \cdot |D|/\text{avgdl})]$$

where $f(q_i, D)$ is the term frequency, avgdl is the average document length over the corpus, and $k_1 \approx 1.2$, $b \approx 0.75$ are the saturation and length-normalisation parameters. IDF is the inverse-document-frequency smoothed against the **full project corpus**, not just the candidate slice — this is the difference between BM25 done well and BM25 done lazily, and it is why tapps-brain builds the in-process IDF index over the entire store.

20.2 Reciprocal Rank Fusion

Lexical and dense channels disagree often — and that disagreement is signal. RRF (Cormack, Clarke, and Büttcher, SIGIR 2009) fuses two or more ranked lists by summing the reciprocal of each candidate's rank in each list:

$$\text{RRF}(d) = \sum_{r \in R} w_r / (k + \text{rank}_r(d))$$

where R ranges over the channels, $\text{rank}_r(d)$ is the document's 1-indexed rank inside channel r , and w_r is a per-query weight chosen deterministically by short heuristics. The constant k defaults to 60 — large enough to keep mid-rank candidates competitive, small enough to honour true top-1 agreement. RRF is rank-only and scale-free: it does not care whether BM25 returned an ts_rank_cd of 0.42 or a vector cosine of 0.18 — it cares which document came first.

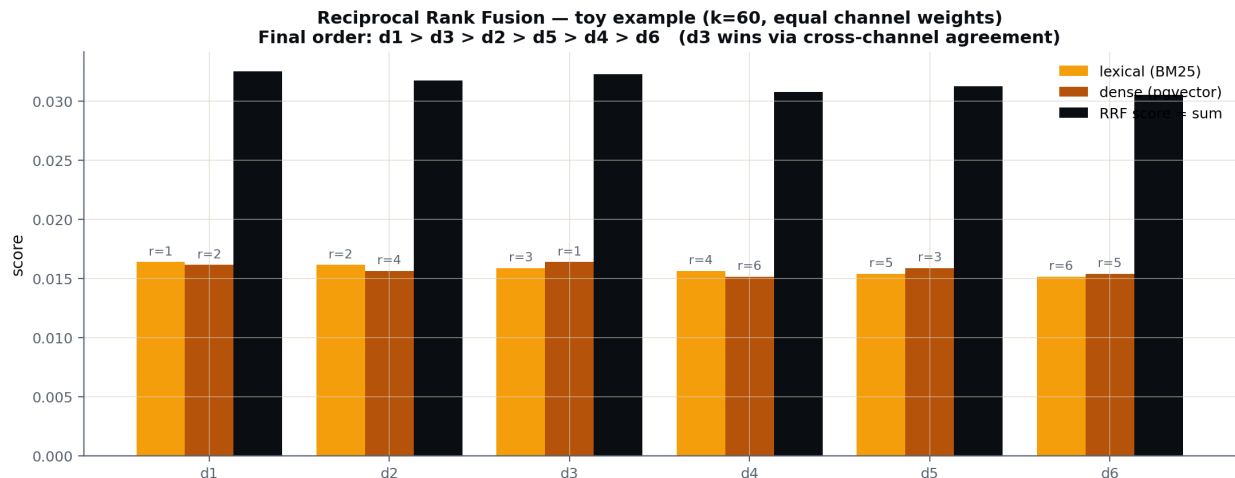


Figure 20.1 — RRF worked example on six documents. Two channels disagree on top-1 (BM25 picks $d1$, vector picks $d3$). RRF picks $d3$ as the fused winner because both channels rank it in their top 3 — the cross-channel agreement, not the score magnitude.

20.3 Composite scoring

After fusion, the system applies a small composite score that turns relevance into a ranking that also respects the human signal — how confident an entry is, how recently it has been touched, and how often it has been useful in the past:

$$\text{score}(d) = 0.40 \cdot \text{relevance}(d) + 0.30 \cdot \text{confidence}(d) + 0.15 \cdot \text{recency}(d) + 0.15 \cdot \text{frequency}(d)$$

(repo-brain defaults; customer-support and home-automation use different weights — see Figure 15.2.) Each term is normalised to [0, 1]. The weights are profile-tunable; the formula is fixed. No LLM in the loop.

20.4 Exponential and power-law decay

Memories lose ranking weight as they age, but only on read (ADR-002 — no wall-clock TTL workers). For an entry of age t with tier half-life h , two decay shapes are supported:

$$\text{Exponential: } \phi_{\text{exp}}(t) = \exp(-\ln 2 \cdot t / h)$$

$$\text{Power-law (FSRS-anchored): } R_{\text{pl}}(t) = (1 + t / h)^{-\beta}, \text{ with } \beta \approx 3.29$$

Both pass through the half-life mark $\phi(h) = R(h) = 0.5$. The exponential is the Ebbinghaus baseline; the power-law is the empirical shape Wixted and FRS converge on. tapps-brain's repo-brain profile uses power-law on all four tiers; other profiles mix.

20.5 FSRS-style stability and difficulty

Beyond tier decay, every entry carries two FSRS-inspired fields: **stability** (how long the entry is expected to remain useful) and **difficulty** (how easily it is forgotten under access pressure). On a useful access, stability grows by a difficulty-dampened multiplier; on a contradicted access, it collapses toward the minimum:

$$S' = S \cdot (1 + \alpha \cdot (1 - D)) \text{ on useful and } S' = \max(S_0, S/2) \text{ on contradicted}$$

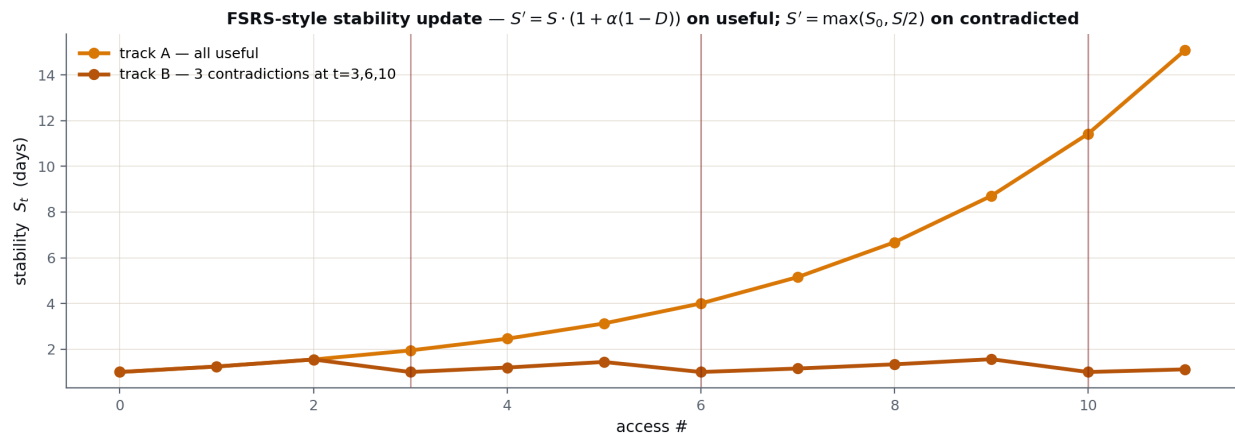


Figure 20.2 — FSRS-style stability under two access tracks: 'all useful' (left curve) versus 'three contradictions' (right curve). Track A grows geometrically; track B collapses to floor on each contradiction and resumes growth from there.

20.6 Bayesian-style confidence update

User and agent feedback events update per-entry confidence using a Bayesian-style smoother. With n_+ positive and n_- negative events and smoothing prior m :

$$c = (n_+ + m \cdot c_0) / (n_+ + n_- + m)$$

where c_0 is the prior (source-trust default for the ingest channel) and m shrinks early evidence toward the prior. The flywheel service maintains this curve in the background; it never blocks a recall.

20.7 Bloom filter false-positive bound

The save path uses a Bloom filter to skip full conflict detection on definitely-not-seen writes. For a filter of m bits, n expected items, and k hash functions:

$$p \approx (1 - e^{-k \cdot n / m})^k$$

Default sizing targets $p \sim 1\%$ at the configured `expected_items` load. A false positive falls through to the full conflict scan; a false negative is impossible, so the filter never causes a missed duplicate.

20.8 Jaccard / TF-IDF for deterministic consolidation

Auto-consolidation merges sufficiently similar entries without any LLM call. Similarity is the maximum of a token-set Jaccard score and a TF-IDF cosine, with an optional vector-cosine tie-breaker when embeddings are present:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \max(J(\mathbf{a}, \mathbf{b}), \cos_{\text{tfidf}}(\mathbf{a}, \mathbf{b})) \text{ with } J(\mathbf{a}, \mathbf{b}) = |\mathbf{T}_a \cap \mathbf{T}_b| / |\mathbf{T}_a \cup \mathbf{T}_b|$$

20.9 EWMA anomaly detector

For a quality signal x_t at step t (e.g. the rolling fraction of recalls that received a useful-rating), the EWMA estimate is:

$$\mu_t = \lambda \cdot x_t + (1 - \lambda) \cdot \mu_{t-1}$$

with smoothing λ in 0.1–0.3. An anomaly fires when μ_t crosses a guard band; the circuit breaker attaches a `quality_warning` to subsequent `RecallResult` objects.

20.10 pgvector HNSW geometry

Dense recall runs over a Hierarchical Navigable Small Worlds graph in pgvector. The index uses $m = 16$ (out-degree per node), `ef_construction = 200` (build-time candidate breadth), and cosine distance. Search is approximate but bounded: expected recall above 95% against exact kNN at a fraction of the latency, with no rebuild after bulk loads.

21. The Knowledge Graph

Why a graph layer, what it stores, and how it stays trustworthy.

Plain text recall answers 'did we decide X?' — but it cannot answer 'how does module X relate to service Y?', 'what other workflows touched this database?', or 'what failed approaches connect to this pattern?'. The Knowledge Graph (EPIC-076) adds a graph layer alongside the memory store, sharing the same Postgres and the same transactional integrity, exposed through five MCP tools.

21.1 Three table families

kg_entities	Named things — modules, services, workflows, concepts. Upserted on (<i>brain_id</i> , <i>entity_type</i> , <i>canonical_name</i>). Carries aliases, metadata, and a confidence score.
kg_edges	Typed relations between entities: <i>uses</i> , <i>signs_with</i> , <i>implemented_by</i> , <i>consumed_by</i> , ... A confidence per edge.
evidence	Quotes / citations that justify edges. Source-typed (<i>doc</i> , <i>chat</i> , <i>commit</i> , <i>test</i>).
experience_events	Monthly RANGE-partitioned append-only log of agent-level events (<i>workflow_completed</i> , <i>tool_called</i> , <i>approach_failed</i> , <i>memory_recalled</i>).

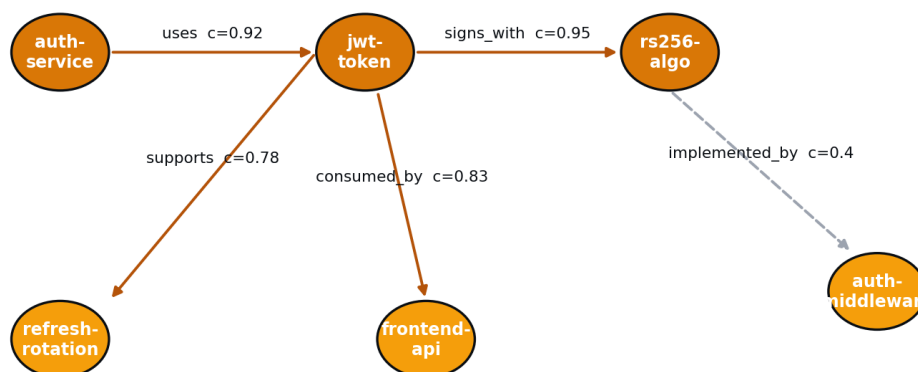
Migration 020 pre-creates 12 monthly partitions of *experience_events* covering through 2027-04. After that, operators must either pre-create further partitions or install *pg_partman* — the default retention is 12 months and is enforced by the operator-side retention policy, not by an in-process job.

21.2 Evidence-required edges (ADR-012)

An edge without evidence is structurally allowed, but its confidence is **capped at 0.4**. This is enforced at the recorder layer: edges with no evidence rows attached cannot exceed 0.4 even if the caller specifies a higher value. The recall ranking then sorts edges by confidence, which makes evidence-backed claims systematically more visible than unsupported ones. The KG never asserts a claim louder than its evidence supports.

Knowledge Graph fragment (auth slice)

solid edges = evidence-backed (uncapped confidence) dashed = no evidence (confidence capped at 0.4 — ADR-012)



evidence: "auth-service uses JWT, signed with RS256 (see ADR-014)" — source: docs/adr/ADR-014.md confidence 0.95

Figure 21.1 — A worked KG slice from an auth subsystem. Solid edges are evidence-backed (uncapped confidence up to ~1.0); the dashed *implemented_by* edge has no evidence and is therefore capped at 0.4. Even though the structural connection exists, the recall ranker treats it as a lower-confidence link.

21.3 The five KG tools — when to reach for each

Tool	Question shape	Cost shape
brain_record_event	'record this workflow + the entities it touched, atomically'	One transaction, up to 5 tables. ~5–15 ms typical.
brain_record_events_batch	'log everything that happened in this workflow run'	Per-event TX or single-TX selectable.
brain_get_neighbors	'what's connected to entity Y?'	Bounded by hop count and per-hop fan-out limit. ~5 ms at 1-hop.
brain_explain_connection	'how does X relate to Y?'	BFS up to 3 hops; returns the path edges and their evidence.
brain_record_feedback	'edge X turned out wrong / right'	Single edge update + FSRS-style stability adjustment.

Fallback rule. If a KG tool returns nothing useful, the agent should fall back to *brain_recall*. The KG is an *additional* surface, not a replacement for text recall. The two together are what gives the agent both *relevance* (what entries match this query) and *relation* (what is connected to what).

22. Data Plane and Security

Tables, indexes, RLS, audit, and the STRIDE-mapped threat model.

Every durable store lives in PostgreSQL (ADR-007). Per-agent isolation is enforced by a *(project_id, agent_id)* composite key on every row, and the brain process connects as the non-privileged *tapps_runtime* role — DML only, no DDL, never bypassing RLS.

22.1 Tables and indexes

private_memories	Per-agent isolated store. Composite PK <i>(project_id, agent_id, key)</i> . <i>tsvector</i> + GIN for lexical, <i>vector(384)</i> + HNSW for dense.
hive_memories	Cross-agent shared store. Namespace-partitioned. <i>tsvector</i> + GIN, <i>vector(384)</i> + HNSW. LISTEN/NOTIFY change feed.
federated_memories	Cross-project hub. Publish/subscribe/pull only.
kg_entities / kg_edges / evidence	The Knowledge Graph plane. Edges without evidence capped at 0.4 (ADR-012).
experience_events	Monthly RANGE-partitioned. 12 months retention by default. Operator-driven pruning.
audit_log	Append-only mutation journal. Indexed on <i>(project_id, agent_id, timestamp DESC)</i> .
diagnostics_history	Periodic EWMA scorecard snapshots.
feedback_events	Per-(project, agent) signal stream.
session_chunks	Native session indexing for episodic recall.
idempotency_keys	Replay protection via <i>X-Idempotency-Key</i> .
agent_registry	Identity, profile, skills, groups, <i>project_root</i> , <i>last_seen</i> .
hive_write_notify	Single-row monotonic <i>revision</i> counter.

22.2 STRIDE threat model

Category	Mitigation
Spoofing	Agent identity session-bound at the store layer; <i>tapps_runtime</i> role is DML-only and RLS-scoped to <i>project_id</i> . DSNs never logged.
Tampering	Safety detection on ingest. Cross-agent writes scoped by the propagation engine. Schema mutations require <i>tapps_migrator</i> . Integrity hash on every entry.

Category	Mitigation
Repudiation	Append-only <i>audit_log</i> with HMAC-v2 integrity hash on every mutation. <i>source_agent</i> and <i>source</i> are non-nullable.
Information disclosure	FORCE ROW LEVEL SECURITY on shared tables. Telemetry policy forbids memory bodies, query text, and PII as span/metric attributes. DSN sanitised in logs.
Denial of service	Per-project entry cap (default 5,000). Bounded <i>psycopg_pool</i> . Rate-limited write path. Deterministic, bounded auto-consolidation.
Elevation of privilege	Separate <i>tapps_runtime</i> and <i>tapps_migrator</i> roles. Operator tools on the separate :8090 MCP with its own admin token. ADR-008 mandates MCP / library / REST parity before any new HTTP route is added.

Encryption at rest is the storage layer's responsibility — Percona *pg_tde* 2.1.2+ or cloud-managed TDE. Application code does not handle keys, never sees key material, and never logs the DSN with credentials.

23. What Works in 2026 — Literature Anchor

The published work tapps-brain inherits from, and the ideas the community has decided to skip.

This chapter maps the design of tapps-brain onto the published work it inherits from. Citations are anchored to author, title, year, and venue. Where the literature is thin or the empirical case is weak, we say so.

23.1 Textbook foundations (still canon in 2026)

- Manning, Raghavan, Schütze, *Introduction to Information Retrieval*, Cambridge University Press, 2008. Chapter 6 (term weighting / tf-idf), Chapter 8 (evaluation: precision, recall, MAP, nDCG), Chapter 11 (probabilistic retrieval and Okapi BM25). The substrate for tapps-brain's lexical channel.
- Robertson and Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in IR 3(4):333–389, 2009. The definitive derivation of BM25 from the Robertson–Sparck Jones framework, including k_1 and b . The $k_1 \approx 1.2$ / $b \approx 0.75$ defaults are still the 2026 production starting point.
- Cormack, Clarke, and Büttcher, *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*, SIGIR 2009. Two pages, one formula, one striking empirical result: RRF beats Condorcet Fuse and individual learning-to-rank methods on TREC and LETOR. *fusion.py* implements it directly.
- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020. The first paper to formalise RAG. Every agent-memory system since, including ours, is a descendant; the 2026 distinction is that systems treat the retriever as the *system of record* and the LM as a stateless reasoner over its output.
- Thakur, Reimers, Rücklé, Srivastava, Gurevych, *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*, NeurIPS 2021. The empirical finding — that BM25 is a hard-to-beat zero-shot baseline and dense models often underperform out-of-distribution — is what justifies keeping a lexical channel in any 2026 production system.
- Anthropic, *Effective Context Engineering for AI Agents*, 2025. Anthropic's explicit recommendation is *agentic memory* — the agent writes structured notes to an external store and re-reads them deliberately — rather than relying on growing context windows. That is the design tapps-brain implements.

23.2 Recent techniques the literature has converged on

- Malkov and Yashunin, *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*, IEEE TPAMI 42(4), 2018. HNSW is the de facto ANN index in 2024–2026. pgvector exposes it; migration 002 uses $m=16$, $ef_construction=200$, $vector_cosine_ops$ — the operating point the paper recommends for cosine-normalised text embeddings.
- Hybrid lexical + dense retrieval, established by BEIR and confirmed by every production benchmark since. Neither BM25 alone (semantic gaps) nor a dense embedder alone (acronyms, identifiers, rare technical terms) is sufficient on agent-memory traffic. tapps-brain commits to hybrid by default; BM25 is non-optional.
- Memory hierarchies — Packer et al., *MemGPT* (UC Berkeley, arXiv 2310.08560, 2023); Park et al., *Generative Agents: Interactive Simulacra of Human Behavior* (UIST 2023); Shinn et al., *Reflexion: Language Agents with Verbal Reinforcement Learning* (NeurIPS 2023). Three independent groups converged on the same architecture: short-term context window backed by an explicit long-term store with consolidation/reflection moving items between tiers. Park et al. introduced the *recency* $\times$ *importance* $\times$

relevance composite score — the exact pattern in tapps-brain's *retrieval.py* (40/30/15/15) and *decay.py*. Reflexion is the reference for episodic memory of failures (*learn_from_failure*).

- **Hippocampus-inspired memory** — Gutiérrez et al., *HippoRAG* (NeurIPS 2024); Zhong et al., *MemoryBank* (AAAI 2024); Xu et al., *A-MEM* (NeurIPS 2025); Yang et al., *Memory³* (arXiv 2407.01178, 2024). HippoRAG combines a KG with Personalized PageRank and outperforms iterative retrieval by 10–30× in cost on multi-hop QA. MemoryBank uses the Ebbinghaus forgetting curve as a design pattern. A-MEM applies Zettelkasten-style dynamic note linking. Shared thesis: memory should be structured, decaying, and indexed both by semantics and by graph relations. tapps-brain's optional KG layer and lazy decay are the direct application; the FSRS reinforcement loop generalises MemoryBank's Ebbinghaus model into a learnable per-item stability parameter.
- **Edge et al., *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, Microsoft Research, arXiv 2404.16130, 2024.** GraphRAG showed that for *global* questions ('what are the main themes?') retrieval over a community-summarised entity graph beats chunk-level RAG. The caveat is that the indexing step is LLM-heavy; tapps-brain keeps KG construction deterministic (rule-based entity/relation extraction in the hot path) and uses the graph as a *navigational* aid alongside BM25 + vector.
- **FSRS — open-spaced-repetition/Free Spaced Repetition Scheduler, building on Woźniak's DSR (Difficulty–Stability–Retrievability) two-component model.** Anki's default scheduler since 23.10 (2024) and the current best-in-class spaced-repetition algorithm. For an agent-memory system, FSRS is the right reinforcement primitive: every successful recall raises stability, every failure raises difficulty. There is no peer-reviewed FSRS journal paper as of early 2026 — we cite it as algorithm-with-implementation, not as a refereed publication.
- **Anthropic, *Introducing the Model Context Protocol*, November 2024 (spec 2024-11-05).** MCP has become the de facto tool surface for agent ↔ external-system interaction in 2025–2026, with SDKs in Python, TypeScript, C#, and Java. tapps-brain exposes its primitives as MCP tools, which is what makes it agent-portable rather than client-locked.
- **Safety filters at ingest.** Prompt-injection-via-RAG is a confirmed production threat class — the 2024 ChatGPT *spAIware* incident and the browsing-context attacks are canonical case studies. Defences include instruction-anomaly detection at ingest, document partitioning, token-masking. tapps-brain's *safety.py* implements ingest-time pattern detection in this lineage. No academic consensus on a complete defence — defence in depth, with the storage layer as one layer of many.

23.3 What the 2025–2026 community has decided *not* to do

- **Continuous fine-tuning on agent traces.** Catastrophic forgetting in continual fine-tuning is now empirically established at scale (arXiv 2308.08747 and follow-ups). Mitigation techniques (EWC, O-LoRA, Titans-style memory modules) trade compute for partial recovery. The simpler external-memory route — what tapps-brain does — sidesteps the problem entirely.
- **Pure vector-only retrieval.** BEIR-class evaluations and 2025 production reports converge: BM25 still beats multi-billion-parameter dense embedders on a non-trivial slice of real queries, especially in technical, code-heavy, or terminology-sensitive domains. Hybrid is strictly dominant. Anyone selling a 'just use embeddings' pitch in 2026 is behind the literature.
- **LLM judges in the request path.** Well-established for offline evaluation (Zheng et al. 2023), but with documented failure modes for online use: nondeterminism, position and length bias, calibration compression at the top of the scale, latency and cost incompatible with hot-path retrieval. tapps-brain's design — deterministic ranking with optional offline LLM-judge replay for flywheel learning — is the conservative reading.

23.4 Reference list

- Anthropic. *Introducing the Model Context Protocol*. 25 November 2024.
- Anthropic. *Effective Context Engineering for AI Agents*. 2025.
- Cormack, G. V., Clarke, C. L. A., Büttcher, S. *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*. SIGIR 2009.
- Edge, D., et al. *From Local to Global: A Graph RAG Approach*. Microsoft Research, arXiv 2404.16130, 2024.
- Gutiérrez, B. J., et al. *HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models*. NeurIPS 2024. arXiv 2405.14831.
- Lewis, P., et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020. arXiv 2005.11401.
- Malkov, Yu. A., and Yashunin, D. A. *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE TPAMI 42(4), 2018. arXiv 1603.09320.
- Manning, C. D., Raghavan, P., Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- Packer, C., et al. *MemGPT: Towards LLMs as Operating Systems*. UC Berkeley, arXiv 2310.08560, 2023.
- Park, J. S., et al. *Generative Agents: Interactive Simulacra of Human Behavior*. UIST 2023.
- Robertson, S., and Zaragoza, H. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR 3(4):333–389, 2009.
- Shinn, N., et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS 2023. arXiv 2303.11366.
- Thakur, N., Reimers, N., Rüklé, A., Srivastava, A., Gurevych, I. *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. NeurIPS 2021 Datasets & Benchmarks. arXiv 2104.08663.
- Xu, W., et al. *A-MEM: Agentic Memory for LLM Agents*. NeurIPS 2025. arXiv 2502.12110.
- Yang, H., et al. *Memory³: Language Modeling with Explicit Memory*. arXiv 2407.01178, 2024.
- Zhong, W., et al. *MemoryBank: Enhancing Large Language Models with Long-Term Memory*. AAAI 2024. arXiv 2305.10250.
- Open-spaced-repetition. *Free Spaced Repetition Scheduler (FSRS)*. Anki default scheduler since 23.10 (2024). Algorithm reference based on Wojcik's DSR memory model.

24. End-to-End Worked Example

Anatomy of one Claude Code session, from the SessionStart hook to the audit log.

The trace below describes a single working session: a backend engineer asks Claude Code to add JWT-with-refresh-rotation authentication to a previously-flat API. Claude has not seen this project before, but the brain has been used for six weeks and holds ~250 entries.

Turn 0 — SessionStart

The harness fires *SessionStart*. The *tapps-brain-session-start.sh* hook prints a *system-reminder* telling Claude to call *brain_recall* with the opening topic. The current branch is *feat/jwt-auth*.

Turn 1 — User opens the session

User: "add JWT with refresh rotation to the API". Claude reads the system-reminder and emits a tool call:

```
{ "tool": "brain_recall", "arguments": { "query": "JWT refresh rotation auth", "max_results": 8 } }
```

The brain returns 7 entries: three *architectural* (a previous JWT decision, the RS256 signing rule, the request-id header convention), three *pattern* (CORS settings, refresh-rotation semantics, structured logging), one *procedural* (the deploy command). All come back ranked, with *recall_diagnostics* showing 12 BM25 candidates and 18 vector candidates fused by RRF (k=60).

Turn 2 — Knowledge graph traversal

Claude notices the recall mentions an entity *auth-middleware*. To find the neighbours:

```
{ "tool": "brain_get_neighbors", "arguments": { "entity_ids": ["auth-middleware"], "hops": 2 } }
```

Returns the path *auth-middleware* → *consumed_by* → *frontend-api* → *depends_on* → *cors-policy* with all edges evidence-backed. Claude now knows the new JWT logic must not break the CORS contract.

Turn 3–N — Implementation

Claude proposes a design, the user accepts, and code is written. No further brain calls in this stretch — the agent has the context it needs.

Turn N+1 — Capture rationale

After acceptance, Claude saves the rationale and shares it with the backend group:

```
{ "tool": "brain_remember", "arguments": { "fact": "Refresh tokens rotate on every use; the previous token is blacklisted in Redis for the remaining TTL.", "tier": "architectural", "share_with": "backend" } }
```

The save propagates to *hive_memories* namespace *backend*, increments *hive_write_notify.revision*, and emits a NOTIFY. *audit_log* records the write with *integrity_hash* and *source_agent=claude-code-wtthornton*.

Turn N+2 — Atomic KG event

Claude records the workflow as an experience event so future agents can find the chain of decisions:

```
{ "tool": "brain_record_event", "arguments": { "event_type": "workflow_completed", "utility_score": 0.9, "payload_json": "{\\\"workflow\\\":\\\"add_auth\\\",\\\"tokens_used\\\":4600,\\\"recall_hits\\\":7}", "entities_json": "[{\\\"entity_type\\\":\\\"workflow\\\",\\\"canonical_name\\\":\\\"add_auth\\\"}]", "edges_json": "[{\\\"predica
```

```
te\":"touches\","src_canonical\":"add_auth\","dst_canonical\":"auth-middleware\"]]"} }
```

Turn N+3 — Finish-task pipeline

The user closes out with `/tapps-finish-task`. The skill runs in three fixed steps: `validate_changed` with the explicit file paths, `checklist(task_type="feature")`, and finally the optional memory save for what was learned at the architectural level. The pipeline fails closed: any step failing aborts the rest.

Turn N+4 — Session end

`SessionEnd` fires. The optional `memory_ingest` hook (if enabled) summarises the transcript and posts it to the brain with tier `context`; the 14-day half-life means this summary will fade quickly unless future sessions reinforce it.

What just changed in the team's collective knowledge

Six weeks ago, the JWT decision existed in nobody's head — the project had flat unauthenticated endpoints. After one session: an architectural memory exists, evidence-backed KG edges connect the new `add_auth` workflow to `auth-middleware`, the backend group's Hive namespace carries the refresh-rotation pattern, and the audit log has a tamper-evident record of who decided what and when. The next agent on this codebase — Claude or human — recalls this in 300 tokens on their first turn.

25. Closing — Business Value Restated

An AI engineering organisation can spend its budget on context every session, or it can spend it once and amortise. tapps-brain is the second option. By converting the ambient knowledge that an agent generates into a ranked, decayed, deterministically retrievable corpus, the system turns repeat work into accumulated capital.

The technical commitments that make this work are conservative on purpose. PostgreSQL is the persistence plane. Determinism is the contract — every recall is reproducible, every merge is logged and undoable, every formula is published. MCP and REST are the transports, with ADR-enforced parity. The layers are separated — instruction (CLAUDE.md / SOUL.md / USER.md), capability (the 65 MCP tools and 20 skills), memory (the brain) — so each can evolve without dragging the others.

Day one

An agent that remembers what it learned yesterday. A team that sees its conventions captured the moment they emerge. An audit trail with an integrity hash on every write. A retrieval pipeline whose ranking can be reproduced from the inputs and explained to an auditor. A deployment topology a single SRE can operate without paging.

Year one

A monotonically improving corpus. Confidence updates from real feedback bring the useful entries to the top of every recall, while exponential or power-law decay and deterministic consolidation quietly remove the rest. Hive lets specialists publish to a group, Federation amortises across projects. The agent stack itself becomes a system that learns its own job.

Persistent memory is not a feature on an agent; it is a strategic asset of the team that operates the agent. tapps-brain is the runtime that turns the asset on.

A. Appendix — Module Dependency Overview

Auto-generated package graph (50 of 138 modules shown).

The dependency graph below is generated from the live source tree. Colour coding: pink = presentation, teal = business / domain, purple = data, grey = infrastructure. The architectural invariant the core enforces is visible in the picture: presentation depends on services, services depend on the memory core, the core depends on protocols, and data adapters sit at the bottom.

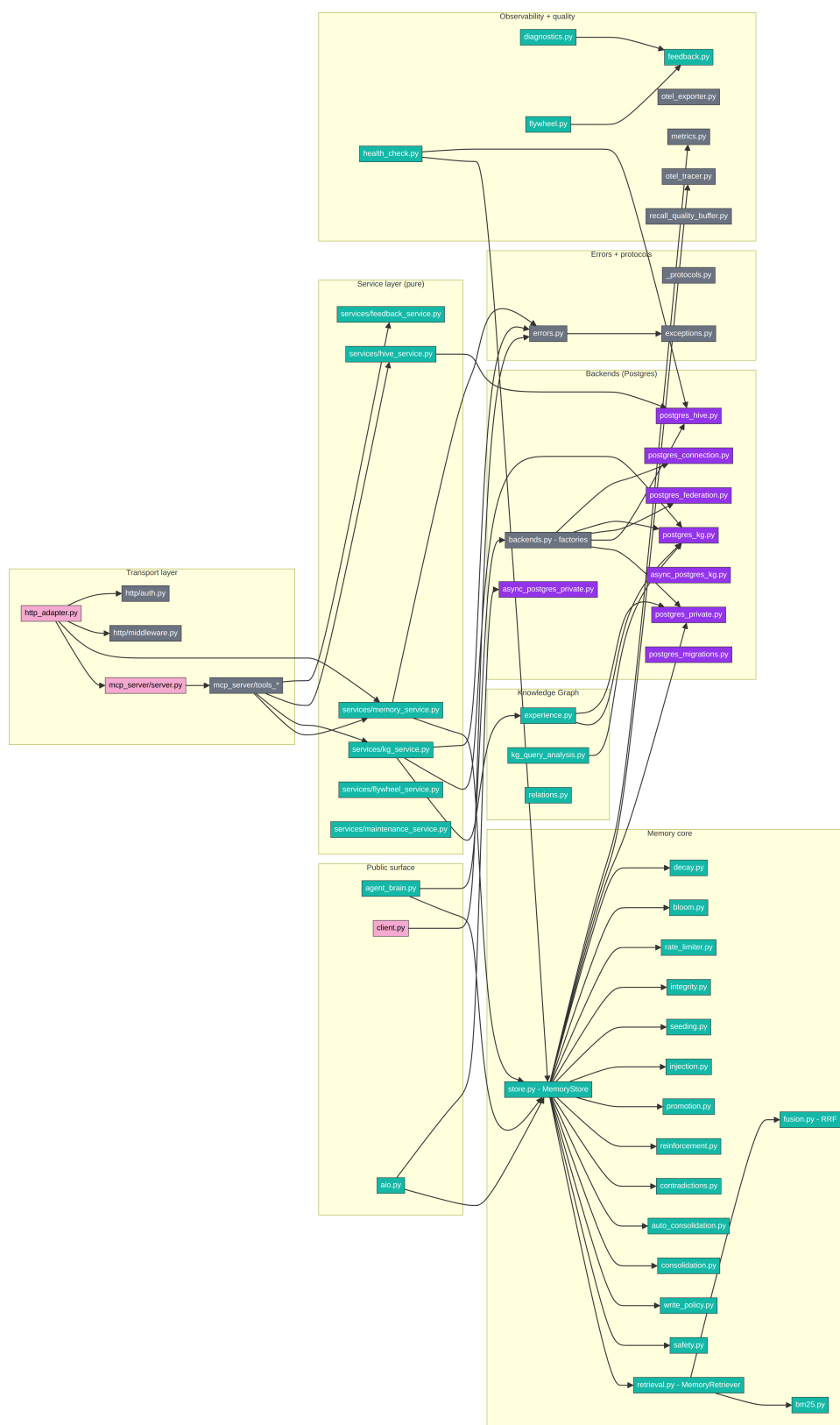


Figure A.1 — Module dependency overview (truncated to 50 of 138 modules).

B. Appendix — Source Verification Log

What was checked against the live source for this edition.

Edition 2.1 was re-verified against the v3.20.x source tree. Every *file:line* citation in the call-tree chapters (16, 17) was confirmed by direct read of the matching *src/tapps_brain/* file. Every numerical constant (HNSW *m*=16, *ef_construction*=200; embedding 384 dims; default *max_entries*=5000; *hive_recall_weight*=0.8; RRF default *k*=60; ADR-012 evidence cap 0.4; FSRS $\beta = \ln(2)/\ln(1+1/k)$ with $k=81/19 \approx 3.29$) was confirmed against the canonical source file. Profile values (tier half-lives, scoring weights, importance-tag multipliers) were confirmed against the YAML in *src/tapps_brain/profiles/*. The SessionStart hook transcript matches *.claude/hooks/tapps-brain-session-start.sh* verbatim.

Counts confirmed by manifest and directory listing:

MCP tools	65 — canonical count from <i>docs/generated/mcp-tools-manifest.json</i>
memory:// resources	8 — read directly from <i>tools_resources.py</i> decorators
Workflow prompts	3 — <i>recall</i> , <i>store_summary</i> , <i>remember</i>
Built-in profiles	6 — repo-brain, customer-support, home-automation, personal-assistant, project-management, research-knowledge
MCP profiles	6 — <i>agent_brain</i> , <i>coder</i> , <i>reviewer</i> , <i>seeder</i> , <i>full</i> , <i>operator</i>
Skills (.claude/skills)	20 — including three linear-* and three ralph-* skill orchestrators
Specialist sub-agents	11 — 7 ralph-* (driver + 6 specialists) and 4 tapps-* (researcher, review-fixer, reviewer, validator)
Hook scripts (.claude/hooks)	20 shell scripts — SessionStart, PreToolUse, PostToolUse, Stop / SessionEnd, PreCompact matchers
Operator-only tools	13 — gated behind the separate <i>tapps-brain-operator-mcp</i> server on :8090
Migration partitions 020	12 monthly partitions covering 2026-05 through 2027-04 (default), then <i>DEFAULT</i> partition; operator-driven extension via <i>pg_partman</i> or pre-creation

Where to find the live values

The fastest way to confirm any number in this document against the running brain is to call *memory://stats* for store-level counts and tier mix, *memory://health* for the retrieval / pool / migration summary, and *memory://agent-contract* for the version, profile, and recall empty-reason taxonomy. These three resources are the canonical live-state probes; anything in a static document — including this one — is a snapshot.